



Universidad Católica Boliviana "San Pablo"
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia



Desarrollo de un Sistema de
Información Organizacional Web
basado en el enfoque de
Procesamiento de Transacciones
(TPS) para la gestión de procesos,
usuarios, transacciones y reportes
para una cafetería

Integrantes:

- Claros Suntura Juan Jose (*Desarrollador FullStack*)
- Lecoña Condori Elias Milan (*Desarrollador FullStack*)
- Maldonado Carvajal Alan Ariel (*Scrum Master*)
- Torrez Calle Alvaro Ariel (*Product Owner*)

Materia: Ingeniería de Software (SIS-213)

Docente: Miguel Angel Pacheco Arteaga

Fecha: 10/04/2026

Universidad: Universidad Católica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS	1
1.1 Introducción	1
1.2 Antecedentes	2
1.2.1 Antecedentes del objeto de estudio	2
1.2.2 Referencias técnicas de otros sistemas TPS	3
1.3 Descripción del objeto de estudio	5
1.4 Descripción de Procesos del Sistema	6
1.4.1 Autenticación y Control de Acceso	6
1.4.2 Gestión del Catálogo (Administrador)	7
1.4.3 Registro de Orden en el POS (Cajero)	7
1.4.4 Procesamiento Transaccional y Cierre (Backend)	8
1.4.5 Generación de Comprobante (Factura/Ticket)	8
1.4.6 Reportes y Arqueo de Caja	8
1.5 Formulación del Problema	8
1.6 Objetivos	9
1.6.1 Objetivo General	9
1.6.2 Objetivos Específicos	9
1.7 Justificación	10
1.7.1 Justificación Técnica	10
1.7.2 Justificación Organizacional	10
1.7.3 Justificación Económica	11
1.8 Límites y Alcances	11
1.8.1 Límites	11
1.8.2 Alcances	12
1.9 Metodología del Proyecto	12
1.9.1 Tipo de estudio	12
1.9.2 Metodología de desarrollo	13

1.9.3 Técnicas	13
1.10 Análisis preliminar del sistema TPS	14
1.11 Propuesta de solución	15
1.12 Cronograma	15
Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS	18
2.1 Sistemas de Información Organizacional	18
2.1.1 Definición	18
2.1.2 Componentes	18
2.1.3 Tipos de sistemas	19
2.2 Sistema de Procesamiento de Transacciones (TPS)	19
2.2.1 Definición	19
2.2.2 Características	20
2.2.3 Funciones	21
2.2.4 Evolución hacia sistemas web	21
2.3 Arquitectura de sistemas web	22
2.3.1 Cliente	22
2.3.2 Servidor	23
2.3.3 API	23
2.3.4 Base de datos	23
2.3.5 Flujo del Sistema	23
2.4 Seguridad en sistemas de información	24
2.4.1 Autenticación	24
2.4.2 Autorización	24
2.4.3 Roles	24
2.4.4 Control de acceso	25
2.5 Base de datos	25
2.5.1 Modelo relacional	25
2.5.2 Integridad	25
2.5.3 Normalización	26

2.5.4	Transacciones	26
2.6	Metodología de desarrollo	26
2.6.1	Scrum	26
Capítulo 3.	MARCO PRÁCTICO	28
3.1	Análisis del sistema	28
3.2	Determinación de requerimientos	30
3.2.1	Requerimientos funcionales	30
3.2.2	Requerimientos no funcionales	34
3.3	Modelado del sistema	36
3.3.1	Historias de Usuario	36
3.3.2	Diagramas UML	48
3.4	Diseño del sistema	51
3.4.1	Arquitectura del sistema — Modelo C4	51
3.5	Diseño de la Base de Datos	55
3.5.1	Paradigma de persistencia	55
3.5.2	Diccionarios de Datos	55
3.5.3	Colección: dishes	61
3.5.4	Colección: categories	62
3.5.5	Relaciones entre colecciones	63
3.6	IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA	64
3.6.1	Módulo de Gestión de Procesos	64
3.6.2	Módulo de Usuarios y Roles	66
3.6.3	Módulo de Transacciones	69
3.6.4	Módulo de Reportes	70
3.7	Capa Backend Funcional	70
3.8	Validación y pruebas del sistema	71
3.9	Desarrollo del prototipo funcional	72
3.10	Documentación de Ingeniería Completa	72
Referencias Bibliográficas		76

Índice de Figuras, Tablas y Diagramas

Figuras

1	Ejemplo de problemas en las cafeterias	2
2	Ejemplo de prueba en el análisis del sistema	29

Tablas

1	Cronograma de Sprints	17
2	Requerimientos funcionales — Gestión de Ventas y Pedidos	30
3	Requerimientos funcionales — Control de Inventario	31
4	Requerimientos funcionales — Gestión de Menú y Productos	32
5	Requerimientos funcionales — Reportes y Toma de Decisiones	33
6	Requerimientos funcionales — Gestión de Proveedores y Compras	33
7	Requerimientos funcionales — Gestión de Personal y Turnos	34
8	Requerimientos no funcionales del Sistema POS	36
9	Historias de Usuario — Módulo de Autenticación y Control de Acceso	37
10	Historias de Usuario — Módulo de Dashboard y Panel de Control	38
11	Historias de Usuario — Módulo de Gestión de Mesas	39
12	Historias de Usuario — Módulo de Gestión del Menú	40
13	Historias de Usuario — Módulo de Toma y Gestión de Órdenes	41
14	Historias de Usuario — Módulo de Cocina (KDS)	42
15	Historias de Usuario — Módulo de Pagos y Facturación	43
16	Historias de Usuario — Módulo de Gestión de Inventario	45
17	Historias de Usuario — Módulo de Gestión de Empleados	46
18	Historias de Usuario — Módulo de Interfaz y Experiencia de Usuario	46
19	Historias de Usuario — Funcionalidades Futuras	48
20	Especificaciones Técnicas del Frontend	53
21	Especificaciones Técnicas del Backend	54

22	Especificaciones Técnicas de la Base de Datos	55
23	Diccionario de datos: Colección Users	56
24	Diccionario de datos: Colección Tables	57
25	Diccionario de datos: Colección Payments	59
26	Diccionario de datos: Colección Orders	61
27	Diccionario de datos: Colección Dishes	62
28	Diccionario de datos: Colección Categories	63
29	Relaciones y Cardinalidad del Modelo de Datos	64
30	Matriz de Control de Acceso por Roles (RBAC)	69

Diagramas

1	Diagrama de Arquitectura MERN	22
2	Diagrama de Casos de Uso — Sistema POS Cafetería	49
3	Diagrama de Clases — Modelo de Dominio del Sistema POS	50
4	Diagrama de Actividades — Proceso de Venta Transaccional	51

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

En el entorno empresarial actual, los Sistemas de Información Organizacional (SIO) se han convertido en un pilar indispensable. Su importancia radica en que permiten a las empresas estructurar, centralizar y gestionar su información para operar con eficiencia y respaldar la toma de decisiones. Dentro de esta jerarquía, el núcleo operativo recae en los Sistemas de Procesamiento de Transacciones (TPS). Estos sistemas han experimentado una profunda evolución tecnológica: pasaron de ser arquitecturas rígidas y lentas de procesamiento por lotes (batch) a convertirse en plataformas dinámicas en línea, capaces de registrar y procesar datos en tiempo real.

Este avance tecnológico responde directamente a la necesidad imperiosa de automatizar procesos. En cualquier negocio, depender de operaciones manuales genera cuellos de botella, errores humanos y una falta total de trazabilidad. Automatizar significa transformar esas tareas repetitivas en flujos de trabajo eficientes. Para lograrlo, el uso de sistemas web se ha consolidado como el estándar ideal en las organizaciones. Desarrollar sobre tecnologías web permite a las empresas implementar herramientas escalables, flexibles y de fácil acceso desde cualquier dispositivo, democratizando soluciones robustas para las pequeñas y medianas empresas.

En este marco, este proyecto consiste en el desarrollo de un Sistema de Información Organizacional Web basado en la evolución del enfoque TPS, que permitirá gestionar procesos, usuarios, transacciones y reportes mediante una plataforma tecnológica accesible y segura. La solución está diseñada para modernizar la gestión operativa de una cafetería en La Paz, Bolivia, reemplazando el uso de libretas de papel por un Punto de Venta (POS) digital que eleve la eficiencia y el control administrativo del establecimiento.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

El objeto de estudio del presente proyecto es una cafetería de tamaño mediano en la ciudad de La Paz, Bolivia, que actualmente opera sus procesos de atención y venta de manera completamente manual. Este escenario, lejos de ser un caso aislado, refleja la realidad predominante en el sector gastronómico local, donde la adopción de tecnologías de gestión sigue siendo incipiente.



Figura 1: Ejemplo de problemas en las cafeterías

En su estado actual, el flujo operativo de la cafetería depende íntegramente de la intervención humana no sistematizada: los pedidos de los clientes son anotados a mano por el personal en libretas o talonarios de papel, la comunicación entre el área de atención y la barra de preparación se realiza de forma verbal o mediante la entrega física de las comandas, y el proceso de cobro se ejecuta mediante cálculos mentales o con el apoyo de calculadoras de escritorio. Al cierre de cada jornada, el arqueo de caja se realiza de forma manual, contrastando el efectivo físico con las anotaciones del día, un proceso propenso a errores y discrepancias.

Esta situación genera un conjunto de problemas operativos concretos y recurrentes que afectan tanto la eficiencia del servicio como la salud financiera del negocio:

- **Pérdida e ilegibilidad de comandas:** El registro manual en papel está ex-

puesto a extravíos, manchas o escritura ilegible, lo que deriva en errores en la preparación de pedidos y conflictos con los clientes.

- **Ausencia de control y auditoría de usuarios:** Al no existir un sistema de registro, resulta imposible determinar qué miembro del personal procesó, modificó o anuló una orden, eliminando cualquier posibilidad de rendición de cuentas.
- **Descuadres de caja recurrentes:** El cobro basado en cálculo mental o en calculadoras simples, sin un sistema que valide automáticamente los totales, genera discrepancias diarias entre los ingresos registrados y el efectivo real.
- **Nula trazabilidad histórica:** La ausencia de una base de datos implica que no existe ningún registro histórico de ventas. La gerencia no puede conocer cuáles son los productos más vendidos, los picos de demanda por hora o los ingresos acumulados por período, privándose de información crítica para la toma de decisiones estratégicas.

1.2.2 Referencias técnicas de otros sistemas TPS

Como parte del análisis de referencia, se relevaron tres sistemas de gestión para cafeterías y restaurantes disponibles públicamente en GitHub. El estudio de estos proyectos permite identificar patrones de diseño comunes, tecnologías adoptadas por la comunidad y brechas funcionales que el presente sistema busca superar.

1. proyecto-cafeteria — ValentinHer (GitHub)

- **Repositorio:** <https://github.com/ValentinHer/proyecto-cafeteria.git>
- **Descripción general:** Sistema de gestión para cafetería desarrollado como proyecto académico. Implementa las operaciones básicas de un punto de venta: registro de productos, toma de pedidos y generación de órdenes.
- **Stack tecnológico:** Aplicación web con arquitectura cliente-servidor, base de datos relacional para la persistencia de productos y transacciones, e interfaz de usuario orientada a la administración del negocio.
- **Funcionalidades identificadas:** Gestión del catálogo de productos (CRUD), registro de pedidos por mesa, y consulta de historial de ventas a

nivel básico.

- **Diferencia con el sistema propuesto:** Este proyecto carece de un sistema de autenticación basado en roles diferenciados (Administrador vs. Cajero), no implementa procesamiento transaccional ACID para garantizar la integridad de los cobros concurrentes, y no genera comprobantes de pago en formato PDF. El sistema propuesto aborda estas limitaciones mediante JWT, sesiones de transacción en MongoDB y el módulo de facturación con PDFKit/jsPDF.

2. Sistema-POS-Restaurantes — ValentinPacheco (GitHub)

- **Repositorio:** <https://github.com/ValentinPacheco/Sistema-POS-Restaurantes.git>
- **Descripción general:** Sistema de Punto de Venta orientado a restaurantes, con enfoque en la gestión de órdenes en sala y el flujo de cobro al cliente. Representa una solución más cercana al dominio del presente proyecto por su naturaleza POS.
- **Stack tecnológico:** Implementación web con separación de capas frontend y backend, manejo de estado de mesas y control de órdenes activas.
- **Funcionalidades identificadas:** Panel de estado de mesas, creación y modificación de órdenes en curso, cálculo de totales y cierre de cuenta por mesa.
- **Diferencia con el sistema propuesto:** Si bien aborda la gestión de mesas y órdenes, el sistema no contempla una arquitectura de microservicios contenerizada con Docker, ni un despliegue en infraestructura *cloud* (AWS/DigitalOcean). Asimismo, el control de acceso por roles y la generación automatizada de reportes financieros con cortes de caja son funcionalidades ausentes que el presente proyecto incorpora de forma nativa.

3. cafeteria-app — FFW4 (GitHub)

- **Repositorio:** <https://github.com/FFW4/cafeteria-app.git>
- **Descripción general:** Aplicación web para la gestión de una cafetería, con foco en la experiencia del operador en el punto de atención. Desarrollada

con un enfoque pragmático orientado a la agilidad en la toma de pedidos.

- **Stack tecnológico:** Aplicación de página única (*SPA*) con componentes reactivos para la interfaz del POS y conexión a un servicio de backend para la persistencia de datos.
- **Funcionalidades identificadas:** Selección de productos por categoría, armado del carrito de compras, asignación de pedidos y registro de ventas.
- **Diferencia con el sistema propuesto:** Esta aplicación no implementa autenticación segura mediante *tokens* JWT ni encriptación de contraseñas con *bcrypt*, exponiendo el sistema a vulnerabilidades de acceso no autorizado. Tampoco cuenta con un módulo de reportes analíticos ni con transacciones ACID multi-documento que garanticen la inmutabilidad de los registros históricos de venta, aspectos críticos en un TPS de producción.

1.3 Descripción del objeto de estudio

La unidad de análisis del presente proyecto es una cafetería de servicio rápido ubicada en la ciudad de La Paz, Bolivia, con capacidad para atender simultáneamente a múltiples mesas. El establecimiento ofrece un menú compuesto principalmente por bebidas calientes y frías (café, infusiones, batidos) y una selección de alimentos de preparación rápida (postres, sándwiches, snacks), atendiendo a una clientela diversa en horario continuo.

Estructura organizacional y actores del sistema:

El personal operativo del establecimiento se organiza en dos roles funcionales claramente diferenciados, que se corresponden directamente con los actores del sistema a desarrollar:

- **Administrador:** Responsable de la gobernanza general del negocio. Gestiona el catálogo de productos (altas, bajas y modificaciones de precios), administra las cuentas del personal operativo y tiene acceso a los reportes de ventas e indicadores financieros.

- **Cajero / Operador POS:** Personal de atención directa al cliente. Su función central es construir y procesar órdenes en la interfaz del punto de venta, asignarlas a la mesa correspondiente y ejecutar el cobro al momento de cerrar la cuenta.

Flujo actual del negocio (situación sin sistema):

El ciclo de servicio actual sigue el siguiente flujo manual: el cliente se ubica en una mesa disponible → el cajero toma el pedido verbalmente y lo anota en la comanda de papel → la comanda se entrega en barra para preparación → los productos son entregados al cliente → al solicitar la cuenta, el cajero suma manualmente los ítems, comunica el total y recibe el pago en efectivo → el dinero se deposita en la caja registradora mecánica.

Flujo propuesto del negocio (con el sistema TPS):

Con la implementación del sistema, el flujo se transforma radicalmente: el cajero selecciona los productos del menú digital interactivo y los asigna a la mesa del cliente en la interfaz POS → el sistema calcula automáticamente el subtotal en tiempo real → al confirmar el cobro, el *backend* procesa matemáticamente la transacción, aplica impuestos y sella el registro de forma inmutable en la base de datos → el sistema genera automáticamente el comprobante de pago (ticket/factura) → la mesa queda marcada como disponible para el próximo cliente. Cada uno de estos eventos queda registrado con fecha, hora, cajero responsable y detalle de productos, garantizando trazabilidad completa y eliminando cualquier posibilidad de descuadre.

1.4 Descripción de Procesos del Sistema

Esta sección describe la forma en que el Sistema TPS-POS será implementado operativamente, detallando los procesos clave que lo componen y cómo cada uno se ejecuta dentro de la arquitectura MERN propuesta.

1.4.1 Autenticación y Control de Acceso

Al ingresar al sistema, el operador introduce sus credenciales (usuario y contraseña) en la pantalla de *login* construida con React.js. El *frontend* envía una solicitud HTTP POST

al *endpoint* `/api/auth/login` del servidor Node.js/Express.js. El *backend* recupera el registro del usuario desde MongoDB, verifica la contraseña mediante la función de comparación de *bcrypt* y, si es válida, genera un JSON Web Token (JWT) firmado que incluye en su *payload* el identificador del usuario y su rol (Administrador o Cajero). Este *token* es devuelto al cliente y almacenado en el estado global de Redux, siendo adjuntado automáticamente en la cabecera *Authorization* de todas las peticiones subsiguientes. Los *middlewares* de Express validan el *token* en cada ruta protegida antes de permitir el acceso a los recursos.

1.4.2 Gestión del Catálogo (Administrador)

El Administrador accede al módulo de gestión de productos desde su panel exclusivo. A través de formularios React, puede crear, editar, activar o desactivar ítems del menú (nombre, precio, categoría, imagen). Cada acción dispara una petición REST al *backend* (POST, PUT o PATCH según corresponda), que valida los datos contra el esquema Mongoose de la colección *Productos* en MongoDB antes de persistir el cambio. Las modificaciones de precio no alteran registros históricos: las órdenes ya cerradas conservan el precio exacto del momento de la venta gracias a la desnormalización del carrito (*cartItems*).

1.4.3 Registro de Orden en el POS (Cajero)

El Cajero opera la interfaz POS táctil. Selecciona una mesa disponible del panel de estados, luego elige productos del menú interactivo organizado por categorías. Cada selección actualiza el estado del carrito en Redux, recalculando subtotales y totales en tiempo real sin consultar el servidor. Una vez completada la orden, el Cajero confirma el método de pago. El *frontend* envía la orden completa (mesa, cajero, *cartItems* con precios actuales, total calculado) al *endpoint* `/api/orders` del *backend*.

1.4.4 Procesamiento Transaccional y Cierre (Backend)

Al recibir la orden, el *backend* inicia una Sesión de Transacción de MongoDB para garantizar las propiedades ACID. Dentro de la transacción atómica se ejecutan en secuencia: (1) validación matemática del total enviado por el cliente, (2) inserción del documento de la orden en la colección Facturas/Órdenes con todos los datos inmutables, (3) actualización del estado de la Mesa asignada de “ocupada” a “disponible”. Si cualquiera de estos pasos falla (por ejemplo, por un corte de red), MongoDB ejecuta un *Rollback* completo, garantizando que no queden “ventas a medias” en la base de datos.

1.4.5 Generación de Comprobante (Factura/Ticket)

Una vez confirmada la transacción, el *backend* devuelve los datos de la orden sellada al *frontend*. React activa automáticamente la opción de generar el comprobante, invocando el módulo de facturación (PDFKit o jsPDF) que construye el ticket en formato PDF con el detalle de los ítems, el total cobrado, la mesa, el cajero y la fecha exacta. El comprobante queda disponible para impresión inmediata desde el navegador.

1.4.6 Reportes y Arqueo de Caja

El Administrador accede al módulo de reportes para consultar el resumen financiero del turno o del período seleccionado. El *backend* ejecuta consultas de agregación (*Aggregation Pipeline*) sobre la colección de órdenes en MongoDB, consolidando ingresos totales, desglose por método de pago, productos más vendidos y ventas por cajero. Los resultados son renderizados en tablas y gráficos en el *frontend* de React, permitiendo al Administrador realizar el arqueo de caja con datos exactos y auditables.

1.5 Formulación del Problema

¿Cómo el desarrollo de un prototipo funcional basado en el procesamiento de transacciones permite proyectar una reducción en los errores operativos y una mejora en el

control de usuarios por roles, asegurando la consistencia de los datos financieros desde una arquitectura web?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un prototipo funcional de un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS), que permita gestionar procesos, usuarios, transacciones y reportes mediante una arquitectura *backend* funcional, con el propósito de proyectar una reducción en los errores operativos y una mejora en el control de usuarios por roles, asegurando la consistencia de los datos financieros desde una arquitectura web.

1.6.2 Objetivos Específicos

- Implementar un módulo de gestión de pedidos en tiempo real que permita a los meseros registrar, modificar y hacer seguimiento del estado de las órdenes (en preparación, servido, pagado) mediante una interfaz táctil dinámica construida con React.js.
- Diseñar e integrar un sistema de control de acceso basado en roles (RBAC) con autenticación segura mediante JSON Web Tokens (JWT), diferenciando los permisos y vistas del Administrador, el Cajero y el Mesero.
- Desarrollar un módulo de administración visual de mesas y reservas que presente un panel de estados en tiempo real, permitiendo a los operadores conocer la disponibilidad y el estado de cada mesa del establecimiento.
- Construir las APIs RESTful del *backend* con Node.js y Express.js, conectadas a una base de datos MongoDB, para soportar todas las operaciones CRUD de los módulos de productos, órdenes, mesas y usuarios.
- Implementar un módulo de facturación y generación de recibos que automatice

el cálculo del cobro total y produzca comprobantes detallados en formato PDF, integrando métodos de pago simulados mediante una pasarela estándar.

- Desplegar el sistema en infraestructura *cloud* (AWS o DigitalOcean) utilizando contenedores Docker para garantizar la portabilidad, disponibilidad y escalabilidad del entorno productivo.

1.7 Justificación

1.7.1 Justificación Técnica

El desarrollo de este prototipo se fundamenta en la implementación de una arquitectura web moderna y de alta disponibilidad, estructurada bajo el entorno Cliente-Servidor empleando el stack MERN. Esta elección garantiza una separación absoluta entre la interfaz de usuario y la lógica de negocio. Se construirá un backend funcional y asíncrono utilizando Node.js y Express.js, optimizado para el procesamiento rápido de múltiples transacciones concurrentes (TPS). Para la persistencia de datos, se utilizará MongoDB, una base de datos orientada a documentos que otorga la flexibilidad necesaria para manejar carritos de compra dinámicos y catálogos escalables. En términos de seguridad, el sistema prescindirá del manejo tradicional de sesiones, implementando en su lugar autenticación sin estado mediante JSON Web Tokens (JWT) y el cifrado irreversible de contraseñas con el algoritmo Bcrypt. Esta infraestructura dota al sistema de una alta escalabilidad técnica, permitiendo futuras integraciones o el despliegue en infraestructuras en la nube (Cloud Hosting) sin necesidad de refactorizar el código base.

1.7.2 Justificación Organizacional

A nivel organizacional, el prototipo representa un salto cualitativo en la gestión de procesos internos de la cafetería. La automatización de transacciones permite estandarizar el ciclo de servicio, eliminando las ambigüedades y retrasos propios de los procesos manuales y la comunicación verbal. Se establecerá un estricto control de usuarios mediante la implementación de un modelo de acceso basado en roles (RBAC), asegu-

rando que los operadores (cajeros) y los tomadores de decisiones (administradores) interactúen únicamente con las interfaces y datos correspondientes a sus responsabilidades. Además, la capacidad del sistema para la generación de reportes automáticos dotará a la gerencia de información estructurada y en tiempo real sobre el rendimiento del negocio, transformando datos aislados en conocimiento estratégico.

1.7.3 Justificación Económica

La viabilidad y necesidad del proyecto se sustentan en el impacto financiero positivo que genera la digitalización. La implementación del TPS asegura una drástica reducción de errores operativos; al automatizar el cálculo de subtotales, impuestos y cambio, se eliminan los recurrentes descuadres de caja diarios. Esto se traduce inmediatamente en la prevención de fugas de capital. Asimismo, la optimización del tiempo por cada transacción permite aumentar la capacidad de atención en horas pico (mayor rotación de mesas), elevando los ingresos. A mediano plazo, estas mejoras operativas confluyen en una notable reducción de costos operativos y administrativos.

1.8 Límites y Alcances

1.8.1 Límites

El alcance del presente proyecto se enmarca dentro de las siguientes restricciones técnicas y operativas:

- El sistema será exclusivamente web, requiriendo un navegador moderno y conexión a la red local o internet para su funcionamiento.
- Se utilizará una base de datos NoSQL (MongoDB) en lugar de una base de datos relacional tradicional, priorizando la agilidad en la estructura de los pedidos (documentos).
- Tendrá un sistema de autenticación de usuarios interno y cerrado; no se permitirá el registro público ni el inicio de sesión con redes sociales.
- No incluirá integración con sistemas externos en esta fase (como plataformas de

delivery de terceros, o pasarelas de impuestos gubernamentales directas).

1.8.2 Alcances

El sistema funcional entregado al finalizar el proyecto permitirá realizar las siguientes operaciones:

- Gestionar procesos operativos centrales, incluyendo la asignación y liberación de mesas de la cafetería.
- Gestionar usuarios y roles, otorgando permisos específicos para la administración del menú o la operación de la caja.
- Registrar transacciones comerciales en tiempo real, almacenando el detalle exacto de cada venta de manera inmutable.
- Generar reportes financieros e históricos de ventas filtrados por fechas o turnos.
- Administrar información maestra del negocio, como el alta, baja y modificación de productos, precios y categorías del catálogo.

1.9 Metodología del Proyecto

1.9.1 Tipo de estudio

La presente investigación se define bajo tres enfoques metodológicos:

- **Aplicado:** Ya que no se limita a la investigación teórica, sino que busca resolver un problema operativo concreto de la cafetería mediante la construcción de una herramienta útil.
- **Tecnológico:** Debido a que el núcleo de la solución requiere la aplicación de ingeniería de software, lenguaje de programación y arquitecturas de sistemas modernos.
- **Descriptivo:** Puesto que requiere analizar, detallar y comprender la naturaleza del flujo de trabajo actual de la organización para poder traducirlo a requerimientos de software.

1.9.2 Metodología de desarrollo

Para la construcción del software se adoptará **Scrum**, un marco de trabajo ágil iterativo e incremental, ideal para proyectos tecnológicos donde los requerimientos pueden evolucionar.

- **Sprint:** El desarrollo se dividirá en ciclos de trabajo cortos y de duración fija (ej. dos semanas), garantizando revisiones periódicas.
- **Product Backlog:** Se manejará una lista priorizada con todos los requerimientos funcionales, técnicos y de interfaz que el sistema POS necesita para operar.
- **Sprint Backlog:** En cada planificación, el equipo seleccionará un subconjunto de tareas del Product Backlog para diseñarlas, codificarlas y probarlas durante ese ciclo.
- **Entregables:** Al finalizar cada Sprint, se presentará un incremento de software funcional (un módulo operativo, como el panel de login o la interfaz de toma de pedidos) listo para ser validado por los interesados.

1.9.3 Técnicas

Para garantizar que el sistema capture fielmente la realidad del negocio y se diseñe con robustez técnica, se aplicarán las siguientes técnicas:

- **Entrevistas:** Se realizarán entrevistas estructuradas a los actores clave (el administrador y el personal de caja). El objetivo es extraer requerimientos funcionales precisos, comprender los cuellos de botella actuales en la toma de pedidos y definir las métricas que la gerencia necesita visualizar en los reportes diarios.
- **Observación:** Se aplicará la observación directa no participante durante las horas pico de la cafetería. Esta técnica es fundamental para mapear el flujo de trabajo real (tiempo de atención por cliente, comunicación entre caja y preparación, y manejo físico del efectivo), identificando fricciones operativas que las entrevistas podrían omitir.

- **Análisis documental:** Se examinarán los registros físicos actuales de la organización (comandas de papel, libretas de contabilidad, inventarios manuales y facturas de proveedores). El análisis de estos documentos es el paso previo a la normalización de datos, permitiendo diseñar los esquemas exactos que conformarán la base de datos (MongoDB).
- **Modelado UML:** Se utilizará el Lenguaje Unificado de Modelado como puente entre la recolección de datos y la codificación. Se elaborarán Diagramas de Casos de Uso para definir la interacción por roles; Diagramas de Actividades para trazar el flujo lógico del Procesamiento de Transacciones (desde el pedido hasta la factura); y Diagramas de Clases para estructurar las entidades del sistema.
- **Modelo C4 (Técnica de Arquitectura):** Complementando a UML, se utilizará el modelo C4 para documentar la arquitectura web. Se elaborarán diagramas de Contexto (el sistema en su entorno), Contenedores (interacción entre React, Node.js y MongoDB), Componentes (controladores y servicios) y Código, facilitando la comprensión técnica del prototipo.

1.10 Análisis preliminar del sistema TPS

El relevamiento inicial demuestra que los **procesos actuales** de la cafetería son enteramente manuales. El ciclo de servicio se basa en la transcripción de pedidos en libretas, cálculos matemáticos mentales y arqueos de caja basados en anotaciones físicas. Los **problemas** derivados de esta operatividad son severos: pérdida constante de comandas, errores humanos en el cobro, lentitud en el servicio y una total falta de auditoría que resulta en discrepancias financieras.

En cuanto a los **usuarios**, el personal opera sin una jerarquía digital; cualquiera puede modificar o anular un registro físico sin dejar rastro. Las **transacciones**, que deberían ser tratadas como eventos inmutables de entrada de capital, carecen de respaldo. Finalmente, la **información** generada por el negocio se pierde al finalizar el día, privando a la gerencia de datos históricos críticos para analizar qué productos se venden más o en qué horarios se requiere más personal.

1.11 Propuesta de solución

Para resolver la problemática descrita, se propone el desarrollo y despliegue de un **Sistema de Información Organizacional Web**, específicamente orientado como un Punto de Venta (POS) transaccional.

- **Arquitectura:** El sistema se construirá bajo el patrón Cliente — Servidor, separando la capa de presentación visual de la capa de procesamiento lógico y acceso a datos, garantizando rendimiento y seguridad.
- **Tecnologías seleccionadas:**
 - **Backend (Servidor y Lógica):** Se desarrollará en **Node.js** utilizando el framework Express.js, creando una API RESTful encargada de la validación matemática, autenticación de usuarios y reglas de negocio.
 - **Base de datos (Persistencia):** Se utilizará **MongoDB** (NoSQL), modelada mediante Mongoose, para almacenar de forma ágil y estructurada los usuarios, catálogos y el historial inmutable de órdenes de venta.
 - **Frontend (Cliente y UI):** Se construirá con la biblioteca **React.js** (junto con HTML, CSS y JavaScript), desarrollando una interfaz de usuario interactiva, dinámica y adaptada a pantallas táctiles, que permita a los cajeros operar con máxima velocidad y mínima fricción.

1.12 Cronograma

El proyecto tiene una duración total de **4 meses (16 semanas)**, organizado en 8 *Sprints* de 2 semanas cada uno bajo el marco Scrum.

Sprint	Semanas	Fase	Actividades principales	Entregable
0	1–2	Inicio y Diseño	Levantamiento de requerimientos, diseño de <i>wireframes</i> UI/UX, modelado de base de datos, configuración del repositorio GitHub y entorno Docker.	<i>Product Backlog</i> , diseño de BD, <i>wireframes</i> aprobados.
1	3–4	<i>Backend</i> – Fundamentos	Configuración de Express.js, conexión MongoDB con Mongoose, modelos de datos (User, Product, Table, Order), sistema de autenticación JWT y <i>middleware</i> de roles.	API de autenticación funcional (registro, <i>login</i> , roles).
2	5–6	<i>Backend</i> – Módulos Core	APIs RESTful para gestión de productos del menú (CRUD), gestión de mesas (CRUD + estados) y gestión de órdenes (crear, actualizar estado).	<i>Endpoints</i> de Products, Tables y Orders documentados.
3	7–8	<i>Frontend</i> – Base y Auth	Configuración de React.js + Redux Toolkit + React Router, pantallas de <i>Login</i> , <i>layout</i> principal y conexión con el API de autenticación.	<i>Frontend</i> base funcional con <i>login</i> y roles.
4	9–10	<i>Frontend</i> – POS	Módulo POS táctil: selección de categorías, productos, cantidad y adición al carrito de órdenes; envío de pedidos al <i>backend</i> .	Interfaz POS funcional conectada al <i>backend</i> .

Sprint	Semanas	Fase	Actividades principales	Entregable
5	11–12	Mesas y Dash-board	Panel visual de estados de mesas, módulo de administración de menú (alta/baja de productos), vista de órdenes activas para cocina.	Gestión de mesas e interfaz de cocina operativa.
6	13–14	Facturación y Pagos	Módulo de generación de facturas en PDF, cálculo automático de totales, integración de métodos de pago simulados, historial de ventas.	Facturación y cierre de órdenes completo.
7	15–16	Cierre: QA y Despliegue	Pruebas funcionales e integración (QA), corrección de <i>bugs</i> , despliegue en AWS/DigitalOcean con Docker, documentación técnica final y capacitación al usuario.	Sistema desplegado en producción y manual de usuario.

Tabla 1: Cronograma de Sprints

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un Sistema de Información Organizacional (SIO) es un conjunto integrado de componentes — personas, procesos, datos, *hardware* y *software* — diseñado para recolectar, almacenar, procesar y distribuir información que apoye la coordinación, el control, el análisis y la toma de decisiones dentro de una organización (Laudon & Laudon, 2020). A diferencia de un simple programa informático, un SIO está profundamente imbricado con los procesos de negocio de la organización: define cómo fluye la información entre los actores, cuándo se captura, cómo se transforma y quién tiene acceso a ella.

En términos más precisos, un SIO transforma datos crudos (ej. el registro de una venta) en información significativa y estructurada (ej. un reporte de ingresos diarios), que a su vez se convierte en conocimiento útil para la gestión (ej. la identificación del turno de mayor rentabilidad). Este proceso de transformación es el núcleo del valor que aportan los SIO a las organizaciones modernas.

2.1.2 Componentes

Todo Sistema de Información Organizacional se articula en torno a seis componentes fundamentales que trabajan de forma interdependiente:

- **Hardware:** La infraestructura física que sustenta el sistema: servidores, terminales de trabajo, dispositivos de red y, en el contexto del presente proyecto, las estaciones de trabajo desde las que el personal operará la interfaz POS.
- **Software:** Los programas y aplicaciones que procesan los datos. Incluye tanto el *software* de sistema (sistema operativo, entorno de ejecución Node.js) como el *software* de aplicación desarrollado a medida (la plataforma POS web).
- **Datos:** La materia prima del sistema. En el contexto de la cafetería, los datos

son las órdenes, los productos, los usuarios, las mesas y las transacciones que el sistema captura y persiste en la base de datos MongoDB.

- **Redes y telecomunicaciones:** La infraestructura de conectividad que permite el acceso concurrente al sistema desde múltiples dispositivos, habilitado por la arquitectura cliente-servidor del proyecto.
- **Procedimientos:** Los protocolos y flujos de trabajo que definen cómo deben interactuar los usuarios con el sistema (ej. el proceso de apertura de turno, la toma de una orden, el cierre de caja).
- **Recursos humanos:** Los actores que operan el sistema. En el proyecto, esto comprende al Administrador y al Cajero, cada uno con roles y permisos claramente delimitados.

2.1.3 Tipos de sistemas

Desde una perspectiva funcional, los SIO se clasifican en distintos tipos según el nivel organizacional al que sirven. Los **Sistemas de Procesamiento de Transacciones (TPS)** operan en el nivel operativo, capturando y procesando las transacciones cotidianas del negocio. Los **Sistemas de Información Gerencial (MIS)** consolidan la información del TPS para generar reportes estructurados destinados a la gerencia media. Los **Sistemas de Soporte a Decisiones (DSS)** asisten en la toma de decisiones complejas mediante análisis de datos y modelos. Los **Sistemas de Información Ejecutiva (EIS)** proveen información estratégica de alto nivel a los directivos. El presente proyecto se enfoca en la implementación de un TPS, que actúa como la base de toda esta pirámide informacional.

2.2 Sistema de Procesamiento de Transacciones (TPS)

2.2.1 Definición

Un Sistema de Procesamiento de Transacciones es un tipo especializado de SIO diseñado para capturar, procesar, validar y almacenar las transacciones operativas de una organización de forma inmediata, confiable y a gran escala (O'Brien & Marakas, 2011).

En el contexto del negocio, una **transacción** se define como cualquier evento discreto que modifica el estado de los datos del sistema y que debe quedar registrado de forma permanente e inalterable: el registro de una venta, la creación de una orden, el cobro de una cuenta o la modificación del catálogo de productos.

2.2.2 Características

Los TPS se distinguen de otros tipos de sistemas de información por un conjunto de atributos técnicos y funcionales que los hacen aptos para el procesamiento operativo de alto volumen:

- **Procesamiento en tiempo real (OLTP):** A diferencia del procesamiento por lotes (*batch*), los TPS modernos procesan cada transacción en el instante en que se produce, actualizando la base de datos de forma inmediata y reflejando el estado actual del negocio en todo momento.
- **Alta confiabilidad y disponibilidad:** Un TPS para un punto de venta debe estar disponible durante todo el horario operativo del negocio. La indisponibilidad del sistema implica la parálisis del servicio al cliente.
- **Integridad transaccional (propiedades ACID):** Toda transacción en un TPS debe cumplir las propiedades de Atomicidad (la transacción se ejecuta completa o no se ejecuta), Consistencia (el sistema pasa de un estado válido a otro estado válido), Aislamiento (las transacciones concurrentes no interfieren entre sí) y Durabilidad (una transacción confirmada persiste incluso ante fallos del sistema) (Elmasri & Navathe, 2015).
- **Manejo de alto volumen de datos estandarizados:** Los TPS están optimizados para procesar grandes cantidades de transacciones simples y repetitivas de forma eficiente, a diferencia de los sistemas analíticos que trabajan con consultas complejas sobre datos históricos.

2.2.3 Funciones

En el contexto específico del presente proyecto, el TPS ejecuta el siguiente ciclo funcional para cada transacción de venta:

1. **Captura de datos de origen:** El cajero construye la orden seleccionando productos del catálogo digital y asignándola a una mesa, introduciendo los datos de la transacción en el sistema mediante la interfaz POS de React.js.
2. **Validación y verificación:** El *backend* (Node.js/Express.js) verifica que el usuario tenga los permisos necesarios (validación JWT), que los ítems existan en el catálogo activo y que la mesa esté disponible.
3. **Procesamiento matemático:** El motor transaccional calcula automáticamente los subtotales por ítem, aplica los impuestos correspondientes y determina el total a cobrar, eliminando el margen de error del cálculo manual.
4. **Actualización de la base de datos:** La transacción se escribe de forma atómica en MongoDB, vinculando el documento de la orden con el cajero responsable, la mesa asignada y los ítems del carrito con sus precios exactos en ese instante.
5. **Emisión del comprobante:** El sistema genera el ticket o factura en formato PDF, disponible para impresión inmediata, y actualiza el estado de la mesa a “disponible”.

2.2.4 Evolución hacia sistemas web

Los TPS han recorrido un largo camino desde las terminales monolíticas de los años setenta. La adopción de arquitecturas web modernas —como la empleada en este proyecto— representa la fase más reciente de esta evolución, caracterizada por tres ventajas fundamentales: **ubicuidad** (el sistema es accesible desde cualquier dispositivo con navegador web en la red local del negocio), **centralización** (todos los datos residen en un único repositorio en la nube, eliminando la dispersión de información), y **escalabilidad** (la arquitectura basada en microservicios y contenedores Docker permite escalar el sistema horizontalmente para absorber incrementos en la carga de trabajo sin rediseñar la arquitectura base).

2.3 Arquitectura de sistemas web

La arquitectura del sistema POS se basa en el modelo Cliente–Servidor, una de las estructuras más utilizadas en aplicaciones web modernas por su capacidad de separación de responsabilidades, escalabilidad y mantenimiento (Sommerville, 2015). Siguiendo este diagrama:

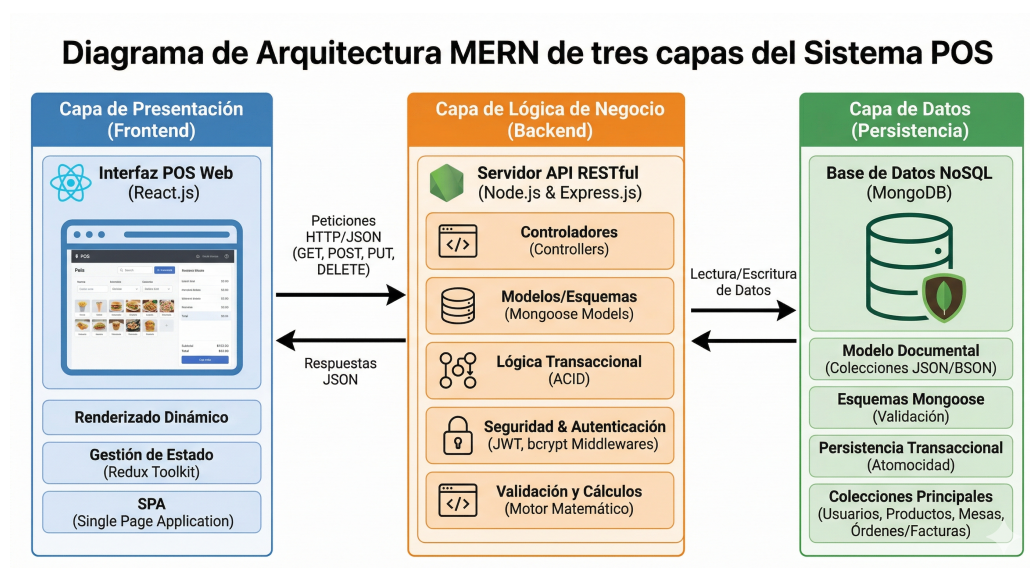


Diagrama 1: Diagrama de Arquitectura MERN

2.3.1 Cliente

El cliente representa la capa de presentación del sistema, encargada de interactuar directamente con el usuario final mediante una interfaz gráfica accesible desde el navegador. En este proyecto, el cliente será desarrollado utilizando React.js, permitiendo:

- Renderizado dinámico de componentes (Virtual DOM).
- Interacciones en tiempo real en el POS.
- Manejo del estado global mediante Redux Toolkit.
- Navegación sin recarga de página (SPA).

Funciones principales:

- Capturar datos de entrada (pedidos, login).

- Mostrar información procesada.
- Enviar solicitudes HTTP al servidor.

2.3.2 Servidor

El servidor constituye la capa de lógica de negocio. Tecnologías: Node.js; Express.js.

Funciones principales:

- Procesamiento de órdenes.
- Validaciones y cálculos.
- Ejecución de la lógica transaccional.

2.3.3 API

La API permite la comunicación entre cliente y servidor mediante HTTP y JSON. Actúa como el puente documentado que estructura y transmite la información bidireccionalmente. Características:

- Métodos estándar: GET, POST, PUT, DELETE.
- Arquitectura RESTful.

2.3.4 Base de datos

Repositorio central donde reposa la persistencia de las entidades. Es el componente responsable de almacenar los datos operacionales a largo plazo para asegurar la durabilidad y disponibilidad de la información de las ventas y el menú.

2.3.5 Flujo del Sistema

1. Usuario interactúa con frontend.
2. Cliente envía petición HTTP a la API.
3. Servidor procesa la solicitud y valida.
4. Se consulta o impacta la base de datos.
5. Servidor responde en JSON.

6. Frontend actualiza la interfaz.

2.4 Seguridad en sistemas de información

Como modelador y encargado de la seguridad arquitectónica, se establece que un sistema de ventas (POS) debe proteger de forma absoluta sus *endpoints* (rutas de API) y la persistencia de datos.

2.4.1 Autenticación

Se descartan las sesiones tradicionales. El sistema implementa JSON Web Tokens (JWT). Tras validar credenciales (contraseñas previamente procesadas con funciones criptográficas unidireccionales de *hash*, como *bcrypt*), el *backend* emite un token firmado (Stallings, 2017). Este token viaja en las cabeceras HTTP de cada petición del cliente, garantizando que el usuario es quien dice ser sin consultar la base de datos reiteradamente.

2.4.2 Autorización

La autorización asegura que un usuario autenticado solo pueda realizar las acciones para las que está facultado. Se ejecuta verificando los niveles de privilegio incrustados y firmados criptográficamente en el token antes de responder a una petición.

2.4.3 Roles

El modelo de datos incluye una propiedad rígida de “Rol” (ej. Administrador, Cajero). Este mecanismo se implementa mediante *Middlewares* (bloques de código intermedios en el *backend*) que desenscriptan el *payload* del token y rechazan con un error 403 (Prohibido) cualquier intento de un Cajero de acceder a las rutas de eliminación de usuarios o reportes gerenciales.

2.4.4 Control de acceso

Tanto a nivel de la interfaz (ocultando botones de configuración a cajeros) como a nivel de capa de datos, se aplican políticas estrictas para evitar inyecciones maliciosas o robo de sesiones, blindando el flujo desde que se presiona “Cobrar” hasta que la información reposa en el disco.

2.5 Base de datos

El diseño de la persistencia de datos constituye el corazón del sistema, siendo responsabilidad directa de la ingeniería de datos modelar la información de la cafetería para que sea escalable, rápida y matemáticamente exacta.

2.5.1 Modelo relacional

Aunque tecnologías modernas como la pila MERN utilicen modelos NoSQL orientados a documentos para optimizar la velocidad transaccional, los principios lógicos relacionales son ineludibles en un TPS (Elmasri & Navathe, 2015). Las entidades maestras se identifican y separan claramente: Usuarios (Personal), Categorías (Clasificación del menú), Productos (Ítems de venta) y Facturas/Órdenes (Registro de la transacción). Se definen llaves referenciales explícitas entre ellas para establecer cardinalidad (ej. un cajero realiza muchas ventas, una orden contiene múltiples productos).

2.5.2 Integridad

Los principios lógicos de integridad se mantienen ineludibles mediante el uso de esquemas de validación estrictos (como *Mongoose* en el caso de la pila seleccionada). Estos esquemas garantizan la exactitud de los tipos de datos ingresados y previenen la inserción de documentos huérfanos o con información financiera incompleta.

2.5.3 Normalización

Se aplican reglas de normalización para evitar anomalías; por ejemplo, la información del perfil del usuario o la descripción detallada de un producto no se repiten innecesariamente. Sin embargo, por requerimientos de diseño de un POS y para proteger la contabilidad, se realiza una desnormalización controlada en las Órdenes: al registrar una venta, el precio exacto actual del producto se copia de forma fija dentro de la orden. Esto garantiza que la información histórica sea inmutable frente a futuros cambios de precios en el catálogo.

2.5.4 Transacciones

En el entorno TPS, una transacción es indivisible. Registrar una venta implica: calcular totales, insertar la orden, asociar el método de pago y actualizar la disponibilidad de la mesa. El motor de la base de datos se configura para garantizar atomicidad (Principios ACID), asegurando que si ocurre un fallo de red a la mitad del proceso, la base de datos ejecute un *Rollback* (reversión completa de los pasos previos), previniendo que existan “ventas a medias” o corrupciones en los arqueos de caja.

2.6 Metodología de desarrollo

2.6.1 Scrum

Es un marco de trabajo ágil para el desarrollo, entrega y mantenimiento de productos complejos, definido en la *Scrum Guide* (Schwaber & Sutherland, 2020). Se fundamenta en tres pilares empíricos: transparencia, inspección y adaptación. Para el Sistema POS de Cafetería, Scrum es la elección metodológica óptima porque permite iterar rápidamente y ajustar requerimientos de acuerdo a la retroalimentación del cliente.

Roles

- **Product Owner:** Es el responsable de maximizar el valor del producto. Sus funciones son definir y mantener el *Product Backlog*; ser el punto de contacto único

con el cliente; y aceptar o rechazar los incrementos funcionales.

- **Scrum Master:** Responsable de que el equipo aplique correctamente Scrum. Facilita las ceremonias, elimina impedimentos y protege al equipo de interrupciones externas.
- **Equipo de Desarrollo:** Equipo autoorganizado responsable de convertir los ítems del *Product Backlog* en un incremento potencialmente entregable (programación, diseño y pruebas).

Artefactos

- **Product Backlog:** Lista única y priorizada de todos los requerimientos funcionales y técnicos necesarios para el sistema.
- **Sprint Backlog:** Conjunto de requerimientos seleccionados para el *Sprint* actual, divididos en tareas concretas a desarrollar por el equipo.
- **Incremento:** La suma de todas las funcionalidades completadas durante el *Sprint*, las cuales deben cumplir con la Definición de Hecho (código probado y validado).

Eventos

- **Sprint Planning (Planificación):** Reunión de inicio donde el equipo define qué se va a entregar y cómo se va a construir el incremento durante el ciclo de trabajo.
 - **Daily Scrum (Reunión diaria):** Reunión breve de sincronización del equipo de desarrollo para evaluar el progreso y exponer bloqueos u obstáculos.
 - **Sprint Review (Revisión):** Demostración del *software* funcional al *Product Owner* y partes interesadas al finalizar el *Sprint* para recoger impresiones.
 - **Sprint Retrospective (Retrospectiva):** Espacio de mejora continua donde el equipo reflexiona sobre sus propios procesos de trabajo de cara a la siguiente iteración.
-

Capítulo 3. MARCO PRÁCTICO

3.1 Análisis del sistema

En base a la recopilación de datos realizada mediante entrevistas al personal y observación directa del flujo operativo de la cafetería, se identifican los siguientes actores, procesos y flujos que el sistema debe digitalizar.

Actores del sistema:

La cafetería opera con tres actores principales que interactúan con el sistema en distintos niveles de privilegio:

- **Administrador:** Actor con control total del sistema. Gestiona el catálogo de productos, categorías y precios; administra usuarios y roles; supervisa el inventario de insumos; genera reportes financieros y de ventas; y toma decisiones estratégicas apoyadas en los indicadores del *dashboard*.
- **Cajero / Operador POS:** Actor de atención directa. Registra ventas, selecciona productos del menú digital, asigna la mesa correspondiente, procesa el cobro y emite el comprobante digital al cliente.
- **Encargado de Cocina:** Actor de apoyo operativo. Recibe notificaciones de pedidos en producción, actualiza la disponibilidad de productos cuando un insumo se agota y gestiona el menú semanal disponible.

Procesos principales identificados:

1. **Proceso de venta:** El cajero construye el pedido seleccionando productos del menú digital → asigna la mesa → el sistema calcula automáticamente el subtotal → se selecciona el método de pago (efectivo o QR) → el *backend* procesa la transacción de forma atómica → se emite el comprobante digital → el inventario se descuenta automáticamente.
2. **Proceso de gestión de inventario:** El administrador registra insumos y productos con su stock inicial → el sistema descuenta automáticamente al registrar cada

venta → cuando un insumo alcanza el nivel mínimo, el sistema emite una alerta
→ el administrador genera la orden de compra vinculada al proveedor correspondiente → al recibir la mercadería, se registra la entrada y el stock se actualiza.

3. **Proceso de gestión de menú:** El encargado de cocina actualiza semanalmente los productos disponibles → asigna precios de venta y costos de producción → agrupa productos por categoría (bebidas calientes, jugos, almuerzos, *snacks*, postres) → marca productos como no disponibles cuando el insumo se agota → gestiona combos y menús del día.
4. **Proceso de reportes:** El administrador consulta el *dashboard* con indicadores en tiempo real → filtra ventas por día, semana o mes → analiza productos más vendidos, horas pico, márgenes por categoría y comparativos históricos → exporta reportes en PDF.

Flujo de datos del sistema:

El flujo de información sigue la dirección: **Entrada (interfaz POS / administración)** → **Procesamiento (backend Node.js/Express.js con validación JWT)** → **Persistencia (MongoDB)** → **Salida (reportes, comprobantes PDF, alertas de stock, dashboard)**. Cada transacción queda vinculada al cajero que la ejecutó, la mesa asignada, los productos con sus precios en ese instante y la fecha y hora exactas, garantizando trazabilidad completa y soporte a auditorías.



Figura 2: Ejemplo de prueba en el análisis del sistema

3.2 Determinación de requerimientos

3.2.1 Requerimientos funcionales

Los requerimientos funcionales expresan lo que el sistema **debe hacer** operativamente. Se organizan por módulo funcional según el análisis del sistema.

4.1 Gestión de Ventas y Pedidos

ID	Descripción del Requerimiento	Actor	Prioridad
RF-01	Registrar cada venta con detalle: producto, cantidad, precio y hora de transacción.	Cajero/Admin	Media
RF-08	Recibir pedidos que ya están hechos o que están en producción.	Admin/Cajero	Media
RF-09	Notificar que el pedido del cliente está preparado o en producción.	Enc. cocina/Admin	Media
RF-11	Emitir comprobantes o tickets digitales al cliente al momento del pago.	Cajero/Admin	Media
RF-12	Soportar múltiples formas de pago: efectivo, código QR.	Cajero/Admin	Alta
RF-13	Visualizar historial de ventas filtrado por día, semana o mes.	Admin	Media
RF-14	Cancelar o modificar pedidos antes de su despacho, con trazabilidad del cambio.	Cajero/Admin	Alta

Tabla 2: Requerimientos funcionales — Gestión de Ventas y Pedidos

4.2 Control de Inventario

ID	Descripción del Requerimiento	Actor	Prioridad
RF-03	Registrar todos los productos e insumos con su stock actual y unidad de medida.	Admin	Alta
RF-15	Descontar automáticamente del inventario al momento de registrar una venta.	Sistema/Admin	Alta
RF-16	Emitir alertas automáticas cuando un insumo alcance el nivel mínimo de stock.	Sistema/Admin	Alta
RF-17	Registrar entradas de mercadería vinculadas a órdenes de compra y proveedores.	Admin	Alta
RF-18	Identificar productos con alta rotación y aquellos con riesgo de desperdicio.	Admin	Media
RF-19	Generar reportes de merma y diferencias de inventario.	Admin	Media

Tabla 3: Requerimientos funcionales — Control de Inventario

4.3 Gestión de Menú y Productos

ID	Descripción del Requerimiento	Actor	Prioridad
RF-05	Crear, editar y desactivar productos del menú de forma rápida y sencilla.	Enc. cocina/Admin	Media
RF-06	Asignar precios de venta y costos de producción a cada producto para calcular el margen real.	Admin	Alta

ID	Descripción del Requerimiento	Actor	Prioridad
RF-07	Agrupar productos por categorías: bebidas calientes, jugos, almuerzos, <i>snacks</i> , postres, etc.	Admin	Alta
RF-10	Elaborar el menú disponible semanal actualizado.	Enc. cocina/Admin	Media
RF-20	Marcar productos como no disponibles cuando el insumo correspondiente se haya agotado.	Enc. cocina/Admin	Alta
RF-21	Gestionar combos o menús del día con precios especiales.	Admin	Media

Tabla 4: Requerimientos funcionales — Gestión de Menú y Productos

4.4 Reportes y Apoyo a la Toma de Decisiones

ID	Descripción del Requerimiento	Actor	Prioridad
RF-04	Reporte de productos más vendidos para identificar los artículos estrella del menú.	Admin	Media
RF-22	Generar reporte de ingresos diarios, semanales y mensuales con comparativos históricos.	Admin	Alta
RF-23	Analizar horas pico para optimizar la asignación de personal por turno.	Admin	Media

ID	Descripción del Requerimiento	Actor	Prioridad
RF-24	Comparar costos versus ingresos por producto y por categoría.	Admin	Alta
RF-25	Mostrar un panel de control (<i>dashboard</i>) con indicadores clave en tiempo real.	Admin	Alta

Tabla 5: Requerimientos funcionales — Reportes y Toma de Decisiones

4.5 Gestión de Proveedores y Compras

ID	Descripción del Requerimiento	Actor	Prioridad
RF-02	Registrar proveedores con sus datos de contacto y condiciones comerciales.	Admin	Media
RF-26	Generar y registrar órdenes de compra vinculadas automáticamente al inventario.	Admin	Alta
RF-27	Mantener historial de compras por proveedor para evaluación y negociación.	Admin	Media
RF-28	Registrar precios de compra históricos para analizar variaciones de costo.	Admin	Media

Tabla 6: Requerimientos funcionales — Gestión de Proveedores y Compras

4.6 Gestión de Personal y Turnos

ID	Descripción del Requerimiento	Actor	Prioridad
RF-29	Registrar datos básicos de empleados: nombre, cargo y turno asignado.	Admin	Media
RF-30	Asociar cada venta al empleado que la realizó para garantizar trazabilidad y control.	Admin	Alta
RF-31	Registrar asistencia y puntualidad por turno.	Admin	Baja
RF-32	Gestionar permisos de acceso diferenciados por rol: administrador y cajero/operario.	Admin	Alta

Tabla 7: Requerimientos funcionales — Gestión de Personal y Turnos

3.2.2 Requerimientos no funcionales

Establecen las restricciones y la forma en cómo debe operar y comportarse estructuralmente la aplicación.

ID	Descripción del Requerimiento	Categoría	Prioridad
RNF-01	El registro de cada venta no debe superar los 5 segundos de respuesta.	Rendimiento	Media
RNF-02	Cada registro de venta se realizará con un <i>token</i> para verificación e historial del administrador.	Confiableidad	Alta
RNF-03	Control de acceso mediante usuario y contraseña con roles diferenciados.	Seguridad	Alta

ID	Descripción del Requerimiento	Categoría	Prioridad
RNF-04	Adición de claves únicas para reservas o llegadas de pedidos, evitando valores duplicados.	Disponibilidad	Media
RNF-05	Arquitectura que permita incorporar nuevos productos, usuarios o sucursales sin rediseño del sistema.	Escalabilidad	Alta
RNF-06	Interfaz intuitiva que permita operar al personal sin capacitación técnica avanzada.	Usabilidad	Alta
RNF-07	Diseño adaptado para uso en pantallas de mostrador.	Usabilidad	Media
RNF-08	El sistema debe estar disponible durante todo el horario de operación sin interrupciones no planificadas.	Disponibilidad	Alta
RNF-09	Las consultas de reportes deben completarse en menos de 10 segundos.	Rendimiento	Media
RNF-10	Protección de datos sensibles frente a accesos no autorizados.	Seguridad	Alta
RNF-11	Respaldo automático de datos para evitar pérdida de información.	Confiabilidad	Alta
RNF-12	Posibilidad de recuperar datos de los últimos 30 días.	Confiabilidad	Alta

ID	Descripción del Requerimiento	Categoría	Prioridad
RNF-13	El sistema debe contar con documentación técnica para facilitar mantenimiento.	Mantenibilidad	Media

Tabla 8: Requerimientos no funcionales del Sistema POS

3.3 Modelado del sistema

3.3.1 Historias de Usuario

Las historias de usuario del Sistema POS Cafetería se elaboraron bajo el formato ágil estándar: *Como* [rol], *quiero* [acción], *para* [beneficio], acompañadas de criterios de aceptación con el esquema *Dado / Cuando / Entonces*. Se presentan agrupadas por módulo funcional, en correspondencia directa con el Product Backlog del proyecto (28 historias de usuario, 122 story points en total).

5.1 Módulo de Autenticación y Control de Acceso

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-01	Registro e inicio de sesión de empleados	Como empleado, quiero registrarme e iniciar sesión con mis credenciales, para acceder al sistema según mi rol asignado.	CA1: Dado que soy un nuevo empleado, cuando completo el formulario con nombre, correo, teléfono, contraseña y rol, entonces mi cuenta es creada y soy redirigido al sistema. CA2: Dado que tengo cuenta existente, cuando ingreso credenciales válidas, entonces recibo un token JWT y soy redirigido a mi vista según mi rol.	Must-have	5	Sprint 1

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-02	Control de acceso por rol	Como administrador, quiero que cada empleado solo acceda a las vistas permitidas para su rol, para garantizar la seguridad y el flujo correcto del sistema.	CA1: Dado que soy barista, cuando intento acceder a la ruta del dashboard, entonces soy bloqueado con un mensaje de acceso denegado. CA2: Dado que soy mesero, cuando inicio sesión, entonces solo veo las opciones de Inicio, Órdenes, Mesas y Menú en la navegación.	Must-have	3	Sprint 1

Tabla 9: Historias de Usuario — Módulo de Autenticación y Control de Acceso

5.2 Módulo de Dashboard y Panel de Control

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-03	Panel de inicio con métricas del negocio	Como mesero o administrador, quiero ver un resumen del estado del negocio al ingresar al sistema, para tomar decisiones rápidas sobre las operaciones del día.	CA1: Dado que inicio sesión exitosamente, cuando se carga la página de inicio, entonces veo métricas de ingresos totales y mesas ocupadas en tarjetas. CA2: Dado que estoy en la página de inicio, cuando hay órdenes recientes, entonces las veo listadas con su estado, mesa y hora de creación.	Must-have	3	Sprint 1

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-21	Métricas globales del sistema	Como administrador, quiero ver estadísticas globales del negocio en el dashboard, para tomar decisiones informadas sobre la operación.	CA1: Dado que accedo a la pestaña de Métricas, cuando se carga la vista, entonces veo tarjetas con total de órdenes, ingresos totales, platos activos, categorías y mesas. CA2: Dado que se crea una nueva orden, cuando actualizo las métricas, entonces los contadores reflejan los datos más recientes.	Should-have	3	Sprint 2

Tabla 10: Historias de Usuario — Módulo de Dashboard y Panel de Control

5.3 Módulo de Gestión de Mesas

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-04	Gestión de mesas del restaurante	Como administrador, quiero crear, editar y eliminar mesas con número, capacidad y color, para mantener actualizada la distribución física de la cafetería.	CA1: Dado que estoy en el módulo de Mesas, cuando creo una nueva mesa con número y capacidad, entonces aparece en la vista con estado Disponible. CA2: Dado que existe una mesa con una orden activa, cuando intento eliminarla, entonces el sistema me impide hacerlo o me advierte del conflicto.	Must-have	3	Sprint 1

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-05	Visualización del estado de las mesas	Como mesero, quiero ver todas las mesas con su estado actual en tiempo real, para saber cuáles están disponibles, ocupadas o en uso.	CA1: Dado que estoy en la página de Mesas, cuando se carga la vista, entonces cada tarjeta muestra número, capacidad, estado y nombre del cliente si está ocupada. CA2: Dado que hay mesas con distintos estados, cuando aplico el filtro <i>En uso</i>, entonces solo se muestran las mesas con órdenes activas.	Must-have	2	Sprint 1

Tabla 11: Historias de Usuario — Módulo de Gestión de Mesas

5.4 Módulo de Gestión del Menú

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-06	Gestión de categorías del menú	Como administrador, quiero crear, editar y eliminar categorías de menú con nombre, color e ícono, para organizar visualmente los productos disponibles.	CA1: Dado que estoy en la sección de Categorías, cuando creo una con nombre, color e ícono, entonces aparece disponible al crear o editar platos. CA2: Dado que una categoría tiene platos asociados, cuando intento eliminarla, entonces el sistema procesa la solicitud o muestra el impacto de la acción.	Must-have	2	Sprint 1

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-07	Gestión de platos del menú	Como administrador, quiero crear, editar y eliminar platos con nombre, precio, categoría e insumos requeridos, para mantener el menú actualizado con los productos disponibles.	CA1: Dado que estoy en la sección de Productos, cuando creo un plato asignándole insumos y cantidades, entonces queda vinculado a esos insumos para el descuento automático de inventario. CA2: Dado que un plato tiene precio o ingredientes desactualizados, cuando lo edito y guardo, entonces los cambios se reflejan inmediatamente en el menú.	Must-have	5	Sprint 1

Tabla 12: Historias de Usuario — Módulo de Gestión del Menú

5.5 Módulo de Toma y Gestión de Órdenes

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-08	Toma de órdenes en el menú	Como mesero, quiero seleccionar una mesa, ingresar los datos del cliente, agregar platos al carrito y confirmar la orden, para registrar el pedido de forma rápida y sin errores.	CA1: Dado que ingreso al módulo de Menú, cuando selecciono mesa, completo datos del cliente y agrego platos, entonces se calcula el total con impuesto y puedo confirmar la orden. CA2: Dado que confirmo una orden en efectivo, cuando el sistema la registra, entonces aparece en la KDS del barista y en la lista de órdenes activas.	Must-have	8	Sprint 1

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-09	Visualización y filtrado de órdenes	Como mesero o administrador, quiero ver todas las órdenes con su estado actual y filtrarlas, para hacer seguimiento del flujo de trabajo de la cocina.	CA1: Dado que estoy en la página de Órdenes, cuando se carga la lista, entonces veo todas las órdenes con cliente, mesa, ítems, total y estado. CA2: Dado que quiero ver solo las órdenes pendientes, cuando selecciono el filtro <i>En Progreso</i>, entonces la lista se actualiza mostrando únicamente esas órdenes.	Must-have	3	Sprint 1

Tabla 13: Historias de Usuario — Módulo de Toma y Gestión de Órdenes

5.6 Módulo de Cocina (KDS)

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-10	Pantalla KDS para barista/cocina	Como barista, quiero ver las órdenes entrantes en mi pantalla de cocina organizadas por estado, para preparar los pedidos en el orden correcto.	CA1: Dado que se crea una nueva orden, cuando aparece en la pestaña <i>Entradas</i> de la KDS, entonces veo el número de mesa, cliente, ítems y cantidad de cada uno. CA2: Dado que comienzo a preparar un pedido, cuando lo muevo a estado <i>Preparando</i>, entonces desaparece de <i>Entradas</i> y aparece en la pestaña correspondiente.	Must-have	5	Sprint 2

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-11	Actualización del estado de una orden	Como barista o mesero, quiero actualizar el estado de una orden (En Progreso, Preparando, Lista, Completada), para comunicar el avance del pedido al equipo.	CA1: Dado que una orden está en estado <i>Preparando</i>, cuando el barista la marca como <i>Lista</i>, entonces el mesero puede verla en la lista de órdenes listas para entregar. CA2: Dado que una orden fue entregada al cliente, cuando se marca como <i>Completada</i>, entonces pasa al historial y la mesa puede liberarse.	Must-have	3	Sprint 2

Tabla 14: Historias de Usuario — Módulo de Cocina (KDS)

5.7 Módulo de Pagos y Facturación

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-12	Pago en línea con Razorpay	Como mesero, quiero procesar pagos en línea mediante la pasarela Razorpay al confirmar una orden, para ofrecer al cliente una alternativa de pago sin efectivo.	CA1: Dado que el cliente elige pago en línea, cuando selecciono ese método y confirmo, entonces se abre el modal de Razorpay con el monto correcto de la orden. CA2: Dado que el cliente completa el pago, cuando es verificado por Razorpay, entonces la orden queda registrada con los datos del pago y se genera la factura.	Must-have	5	Sprint 2

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-13	Generación de factura / comprobante	Como mesero, quiero generar una factura al completar una orden, para entregar al cliente un comprobante detallado de su consumo.	CA1: Dado que una orden fue pagada, cuando accedo a la opción de factura, entonces se muestra un documento con ítems, cantidades, subtotal, impuesto y total. CA2: Dado que estoy viendo la factura generada, cuando selecciono imprimir, entonces el sistema abre el diálogo de impresión con el formato correcto.	Must-have	3	Sprint 2

Tabla 15: Historias de Usuario — Módulo de Pagos y Facturación

5.8 Módulo de Gestión de Inventario

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-14	Gestión de insumos del inventario	Como administrador, quiero agregar, editar y eliminar insumos con stock, unidades y umbrales mínimos y máximos, para controlar con precisión los materiales disponibles en la cafetería.	CA1: Dado que estoy en la página de Insumos, cuando agrego un nuevo insumo con nombre, unidad, stock actual, stock mínimo y costo unitario, entonces aparece en la lista con su estado de stock calculado. CA2: Dado que el stock de un insumo cambió, cuando lo edito y guardo, entonces el estado (Crítico/Bajo/Normal/Abundante) se recalcula automáticamente.	Must-have	5	Sprint 2

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-15	Descuento automático de inventario al crear orden	Como administrador, quiero que al registrar una orden el sistema descuente automáticamente los insumos utilizados, para mantener el inventario actualizado sin intervención manual.	CA1: Dado que se confirma una orden con platos con insumos asociados, cuando el sistema la registra, entonces el stock de cada insumo se reduce según la cantidad usada por plato. CA2: Dado que se registra el descuento, cuando reviso el historial de consumos, entonces aparece un registro con fecha, cantidad y descripción de la orden.	Must-have	5	Sprint 2
HU-16	Alertas de stock crítico y bajo	Como administrador, quiero recibir alertas visuales cuando un insumo llega a nivel crítico o bajo, para reabastecerlo antes de que afecte las operaciones.	CA1: Dado que el stock cae por debajo del mínimo, cuando accedo a la página de Insumos, entonces ese insumo aparece destacado con un indicador <i>Crítico</i> o <i>Bajo</i>. CA2: Dado que hay múltiples insumos con stock crítico, cuando veo el panel de alertas, entonces se listan todos los insumos que requieren atención con su nivel actual.	Must-have	3	Sprint 2
HU-17	Reabastecimiento manual de insumos	Como administrador, quiero registrar el reabastecimiento de un insumo indicando la cantidad recibida, para actualizar el stock y mantener el historial de entradas.	CA1: Dado que recibo una entrega de insumos, cuando ingreso la cantidad a reponer en el formulario, entonces el stock del insumo aumenta por esa cantidad. CA2: Dado que se registra un reabastecimiento, cuando reviso las métricas, entonces el insumo ya no aparece en el panel de alertas si superó el umbral mínimo.	Must-have	2	Sprint 2

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-18	Registro manual de consumo de insumos	Como administrador, quiero registrar consumos de insumos fuera de órdenes (mermas, pruebas, limpieza), para tener un historial de consumo completo y preciso.	CA1: Dado que necesito registrar un consumo especial, cuando ingreso la cantidad y descripción, entonces el stock baja y el evento queda en el historial del insumo. CA2: Dado que se registra el consumo, cuando reviso el detalle del insumo, entonces veo el registro con fecha, cantidad, costo estimado y descripción.	Should-have	2	Sprint 2
HU-19	Métricas de gasto diario en inventario	Como administrador, quiero ver un gráfico de gasto en inventario de los últimos 7 días, para entender los costos operativos y detectar anomalías.	CA1: Dado que estoy en la página de Insumos, cuando accedo a la sección de métricas, entonces veo un gráfico de barras con el gasto por día de los últimos 7 días. CA2: Dado que el gasto de un día es inusualmente alto, cuando reviso el gráfico, entonces puedo identificar fácilmente el día y compararlo con el promedio.	Should-have	3	Sprint 2
HU-23	Búsqueda y filtrado de insumos	Como administrador, quiero buscar insumos por nombre y filtrarlos por estado de stock, para encontrar rápidamente los insumos que necesito gestionar.	CA1: Dado que hay muchos insumos registrados, cuando escribo parte del nombre en el buscador, entonces la lista se filtra en tiempo real mostrando solo los coincidentes. CA2: Dado que quiero ver solo los insumos críticos, cuando aplico el filtro <i>Crítico</i>, entonces la lista muestra únicamente los insumos con ese estado.	Should-have	2	Sprint 2

Tabla 16: Historias de Usuario — Módulo de Gestión de Inventario

5.9 Módulo de Gestión de Empleados

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-20	Gestión de empleados desde el dashboard	Como administrador, quiero ver la lista de empleados activos y eliminar cuentas desde el panel de administración, para mantener el control del acceso al sistema.	CA1: Dado que estoy en la sección de Empleados, cuando se carga la lista, entonces veo el nombre, correo, teléfono y rol de cada empleado. CA2: Dado que un empleado ya no trabaja en la cafetería, cuando elimino su cuenta, entonces no puede volver a iniciar sesión con esas credenciales.	Should-have	2	Sprint 2

Tabla 17: Historias de Usuario — Módulo de Gestión de Empleados

5.10 Módulo de Interfaz y Experiencia de Usuario

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-22	Modo oscuro / claro en la interfaz	Como cualquier usuario, quiero alternar entre modo oscuro y modo claro, para adaptar la interfaz a las condiciones de iluminación del entorno donde trabajo.	CA1: Dado que estoy usando el sistema en un área poco iluminada, cuando activo el modo oscuro desde la barra de navegación, entonces toda la interfaz cambia a paleta oscura sin recargar la página. CA2: Dado que cambié al modo oscuro, cuando cierro y vuelvo a abrir el navegador, entonces el sistema recuerda mi preferencia y mantiene el tema seleccionado.	Should-have	2	Sprint 2

Tabla 18: Historias de Usuario — Módulo de Interfaz y Experiencia de Usuario

5.11 Funcionalidades Futuras (No Asignadas)

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-24	Notificaciones de stock bajo en tiempo real	Como administrador, quiero recibir notificaciones emergentes cuando se detecta un insumo por debajo del mínimo después de una orden, para reaccionar de inmediato sin revisar el inventario manualmente.	CA1: Dado que se confirma una orden y el descuento deja un insumo bajo el mínimo, cuando la orden es procesada, entonces el sistema muestra una notificación <i>toast</i> con el nombre del insumo afectado. CA2: Dado que recibo la notificación, cuando la descarto, entonces puedo continuar con la operación normal sin interrupciones.	Could-have	3	No asignado
HU-25	Exportación del historial de órdenes a CSV	Como administrador, quiero exportar el historial de órdenes a un archivo CSV, para analizarlo en herramientas externas de reportes.	CA1: Dado que estoy en la sección de órdenes del Dashboard, cuando hago clic en <i>Exportar CSV</i>, entonces se descarga un archivo con todas las órdenes con fecha, mesa, cliente, ítems y total. CA2: Dado que aplico un filtro por fecha antes de exportar, cuando descargo el archivo, entonces solo contiene las órdenes del rango seleccionado.	Could-have	5	No asignado
HU-26	App móvil nativa para meseros	Como mesero, quiero una aplicación móvil nativa (iOS/Android) para tomar órdenes desde mi celular, para no depender de una tablet o computadora fija en el salón.	CA1: Dado que descargo la app en mi teléfono, cuando inicio sesión, entonces tengo acceso completo al módulo de mesas y toma de órdenes con interfaz optimizada para pantalla pequeña. CA2: Dado que tomo una orden desde el móvil, cuando la confirmo, entonces aparece en la KDS del barista igual que si se hubiera creado desde la web.	Won't-have	13	No asignado

ID	Título	Historia de Usuario	Criterios de Aceptación	Prioridad	SP	Sprint
HU-27	Programa de fidelización de clientes	Como administrador, quiero registrar clientes frecuentes y acumular puntos por compra, para ofrecerles descuentos o beneficios como estrategia de retención.	CA1: Dado que un cliente realiza su quinta compra, cuando el mesero ingresa su teléfono al tomar la orden, entonces el sistema muestra los puntos acumulados y si aplica algún beneficio. CA2: Dado que un cliente canjea sus puntos, cuando el mesero aplica el descuento, entonces la factura refleja el descuento y los puntos son deducidos.	Won't-have	8	No asignado
HU-28	Integración con plataformas de delivery	Como administrador, quiero recibir órdenes de plataformas externas directamente en el POS, para centralizar todas las órdenes en un solo sistema.	CA1: Dado que llega un pedido de una plataforma de delivery, cuando el sistema lo recibe vía webhook, entonces aparece automáticamente en la KDS con la etiqueta de la plataforma de origen. CA2: Dado que la plataforma cambia el estado de la orden, cuando el sistema sincroniza, entonces el estado en el POS se actualiza para reflejar el cambio.	Won't-have	21	No asignado

Tabla 19: Historias de Usuario — Funcionalidades Futuras

3.3.2 Diagramas UML

Los diagramas UML del sistema han sido elaborados como evidencia gráfica del diseño funcional y estructural. Se incluyen los siguientes tipos de diagrama:

Diagrama de Casos de Uso:

El diagrama de casos de uso representa las interacciones entre los tres actores del sistema —Administrador, Cajero/Mesero y Barista/Cocina— y las funcionalidades que cada uno puede ejecutar dentro del Sistema POS. El Administrador tiene acceso a la

gestión de empleados, catálogo, inventario y reportes; el Cajero opera el punto de venta (toma de órdenes, pagos y facturación); y el Barista interactúa con la pantalla KDS para gestionar el estado de los pedidos en producción.

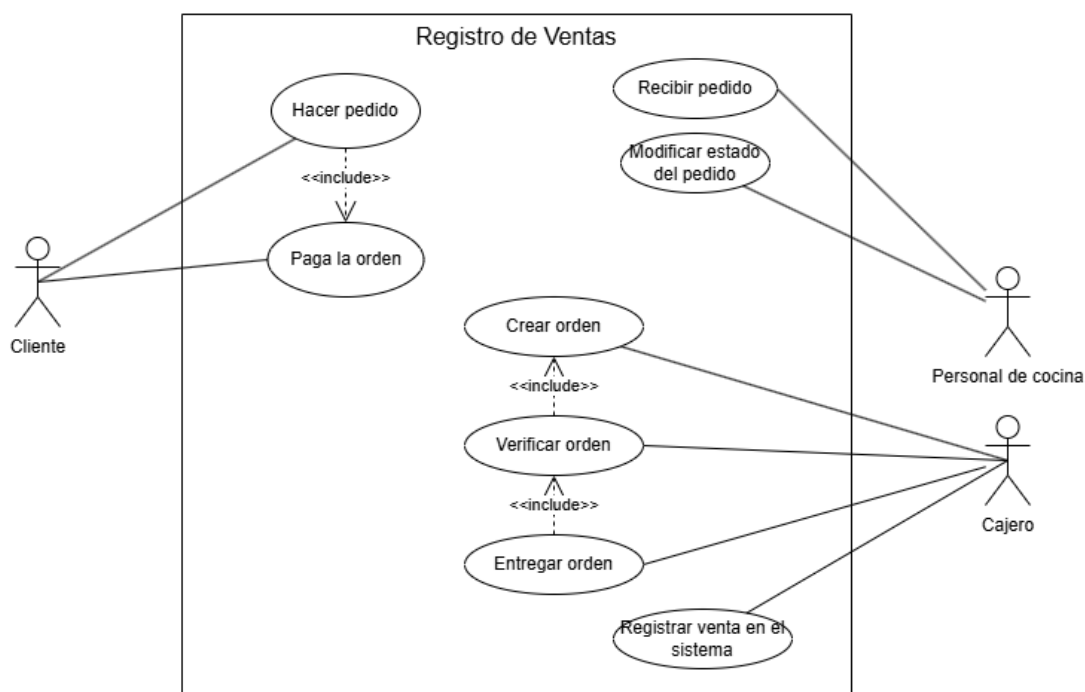


Diagrama 2: Diagrama de Casos de Uso — Sistema POS Cafetería

Diagrama de Clases:

El diagrama de clases expone la estructura estática del modelo de datos del sistema, mapeando las seis entidades principales del dominio: User, Category, Dish, Table, Order y Payment, complementadas por la entidad Insumo para el control de inventario. Se detallan los atributos tipados de cada clase, sus métodos de negocio y las relaciones de cardinalidad (1:N entre usuarios y órdenes, 1:N entre categorías y platos, 1:1 entre órdenes y pagos, y 1:1 entre mesas y su orden activa).

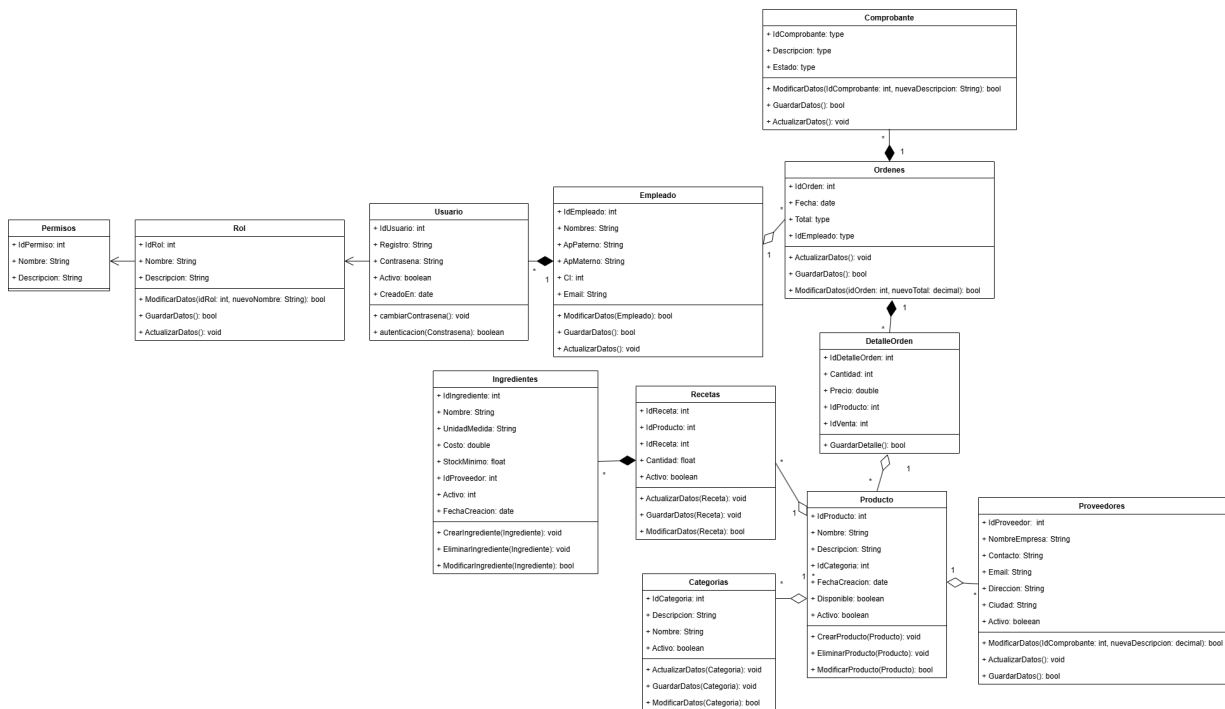


Diagrama 3: Diagrama de Clases — Modelo de Dominio del Sistema POS

Diagrama de Actividades:

El diagrama de actividades modela el flujo completo del proceso de venta transaccional (TPS), desde el inicio de sesión del cajero hasta el cierre y liberación de la mesa. Se divide en tres swimlanes que representan las responsabilidades paralelas del Cajero/Mesero, el Sistema Backend y el Barista/Cocina. Se evidencian los puntos de decisión críticos: validación de credenciales JWT, éxito o fallo de la transacción ACID (con rollback automático ante fallos), y la generación del comprobante PDF al completar el cobro.

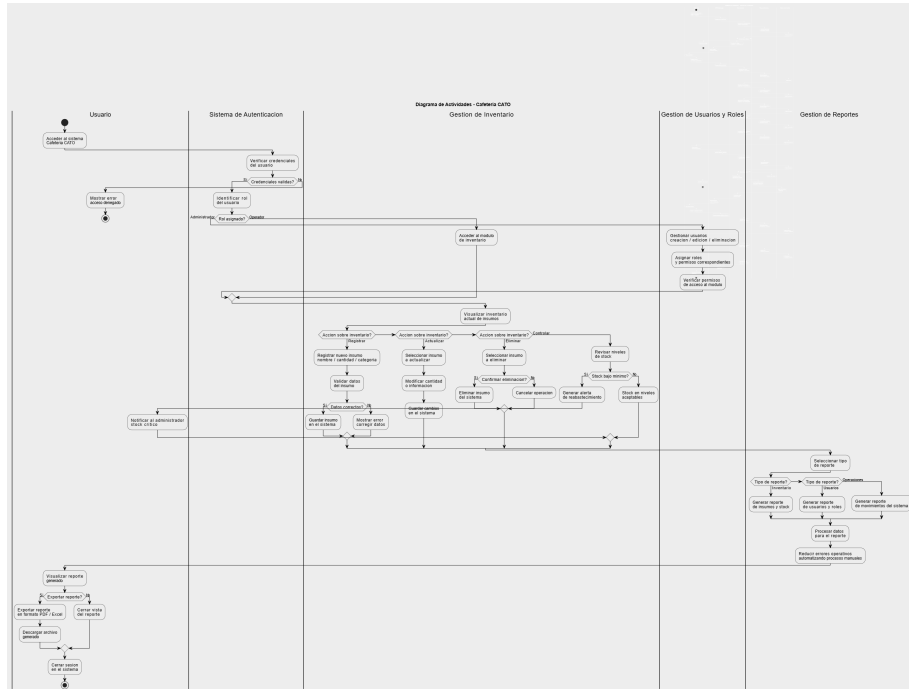


Diagrama 4: Diagrama de Actividades — Proceso de Venta Transaccional

3.4 Diseño del sistema

3.4.1 Arquitectura del sistema — Modelo C4

El diseño arquitectónico del Sistema POS para la cafetería se documenta utilizando el **Modelo C4** (*Context, Containers, Components, Code*), un estándar de representación jerárquica que permite comunicar la arquitectura de software a diferentes audiencias —desde la gerencia hasta los desarrolladores— con el nivel de detalle apropiado para cada una (Brown, 2018).

Nivel 1 — Diagrama de Contexto (*System Context*) El Diagrama de Contexto es la vista de más alto nivel. Su propósito es mostrar el sistema como una caja negra y situar al lector en el entorno donde opera: quiénes interactúan con él y con qué sistemas externos se conecta.

Actores (usuarios del sistema):

- **Administrador:** Interactúa con el sistema a través de un navegador web en su

estación de trabajo. Sus acciones se centran en la gestión del catálogo de productos y usuarios, y en la consulta de reportes financieros.

- **Cajero / Operador POS:** Interactúa con el sistema a través de la interfaz táctil del POS desde la pantalla del punto de atención. Registra órdenes, gestiona el estado de las mesas y procesa los cobros de cada turno.

El sistema central:

- **Sistema POS Web — Cafetería (La Paz, Bolivia):** Plataforma web construida sobre la arquitectura MERN, que centraliza la gestión de transacciones, usuarios, productos y mesas del establecimiento.

Sistemas externos:

- **Módulo de Simulación de Pagos (Mock Gateway):** Sistema encargado de emular el comportamiento de una pasarela de pagos real. Permite registrar y confirmar transacciones mediante tarjetas de crédito o códigos QR de forma controlada, validando la lógica de negocio del POS sin necesidad de interactuar con entidades bancarias externas. Si bien el sistema fue diseñado siguiendo los estándares de integración de pasarelas como Razorpay o Libelula, la versión actual del prototipo utiliza un Entorno de Simulación (Mock Gateway). Esta decisión permite validar el flujo completo de la lógica de negocio (apertura, cobro y cierre de mesa) sin incurrir en costos transaccionales ni depender de conectividad externa durante las pruebas de estrés del sistema.
- **Servicio de Hosting Cloud (AWS EC2 / DigitalOcean):** Infraestructura de nube donde se despliegan los contenedores Docker que alojan la API y la base de datos del sistema.

Relaciones clave en este nivel:

El *Administrador* y el *Cajero* acceden al Sistema POS a través del protocolo HTTPS desde sus respectivos navegadores. El Sistema POS se comunica con la *Pasarela de Pagos* mediante llamadas HTTP/REST para procesar cobros electrónicos, y reside desplegado en el *Servicio de Hosting Cloud*.

Nivel 2 — Diagrama de Contenedores (*Containers*) El Diagrama de Contenedores descompone el sistema en sus bloques tecnológicos desplegados de forma independiente. Cada contenedor es una unidad ejecutable (una aplicación, un servicio, una base de datos) con una tecnología concreta.

Contenedor 1 — Aplicación Web SPA (Frontend)

Atributo	Detalle
Tecnología	React.js 18 + Redux Toolkit + React Router DOM
Tipo	Single Page Application (SPA) — ejecutada en el navegador
Responsabilidad	Renderizar la interfaz del POS, el panel de administración y los reportes. Gestionar el estado global de la sesión y del carrito de compras.
Comunicación	Envía peticiones HTTP/REST en formato JSON a la API Backend a través de axios. Recibe el JWT del backend y lo adjunta en la cabecera Authorization de cada petición subsiguiente.

Tabla 20: Especificaciones Técnicas del Frontend

Contenedor 2 — API REST (Backend)

Atributo	Detalle
Tecnología	Node.js 20 LTS + Express.js 4
Tipo	Servidor de API RESTful — desplegado en contenedor Docker (AWS EC2)

Atributo	Detalle
Responsabilidad	Exponer los <i>endpoints</i> REST (/api/user, /api/order, /api/table, /api/payment, /api/category, /api/dish, /api/metric). Ejecutar la lógica de negocio, validaciones, cálculos transaccionales y control de acceso por roles mediante <i>middlewares</i> JWT.
Comunicación	Recibe peticiones HTTPS del Frontend. Lee y escribe documentos en MongoDB a través del ODM Mongoose. Invoca la API de la Pasarela de Pagos externa cuando se procesa un cobro electrónico.

Tabla 21: Especificaciones Técnicas del Backend

Contenedor 3 — Base de Datos (MongoDB)

Atributo	Detalle
Tecnología	MongoDB 7 (desplegado en contenedor Docker o MongoDB Atlas)
Tipo	Base de datos NoSQL orientada a documentos
Responsabilidad	Persistir de forma duradera todos los documentos del sistema: usuarios (users), productos (products), mesas (tables), órdenes (orders) y pagos (payments).

Atributo	Detalle
Comunicación	Solo es accedida directamente por el Backend API a través del driver Mongoose. No expone puertos públicos; es accesible únicamente dentro de la red privada del entorno Docker.

Tabla 22: Especificaciones Técnicas de la Base de Datos

3.5 Diseño de la Base de Datos

3.5.1 Paradigma de persistencia

El sistema adopta **MongoDB** como motor de base de datos NoSQL orientado a documentos, gestionado a través del ODM (Object Document Mapper) **Mongoose**. Esta elección responde a los requisitos de un TPS moderno: flexibilidad en el esquema para los ítems de la orden (cuyo número varía por transacción), alta velocidad de escritura para el registro de ventas en tiempo real, y soporte nativo para transacciones ACID multi-documento mediante *Sessions* en MongoDB 4+.

A pesar del paradigma documental, el diseño lógico preserva los principios de integridad referencial relacionales mediante el uso de *ObjectId* y la configuración *ref* de Mongoose, que permiten realizar operaciones de *populate* (equivalentes a JOIN) entre colecciones relacionadas.

3.5.2 Diccionarios de Datos

Colección: users Almacena los registros del personal operativo y administrativo con acceso al sistema.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.
name	String	Sí	Nombre completo del usuario.	Mínimo 3 caracteres.
email	String	Sí	Dirección de correo electrónico.	Validado con expresión regular. Indexado como unique: true.
phone	Number	Sí	Número de teléfono de contacto.	Debe contener 10 dígitos.
password	String	Sí	Contraseña de acceso al sistema.	Almacenada como <i>hash</i> irreversible con Bcrypt (coste: 10). Nunca en texto plano.
role	String	Sí	Rol funcional asignado al usuario.	Valores: "Admin" o "Cashier". Determina permisos en los <i>endpoints</i> .
createdAt	Date	Sí	Fecha y hora de creación del registro.	Generado automáticamente por timestamps de Mongoose.
updatedAt	Date	Sí	Fecha y hora de la última modificación.	Actualizado automáticamente por timestamps de Mongoose.

Tabla 23: Diccionario de datos: Colección Users

Colección: tables Gestiona la información de las mesas físicas del establecimiento y su estado operativo en tiempo real.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.
tableNo	Number	Sí	Número identificador de la mesa.	Configurado como unique: true. No pueden existir dos mesas con el mismo número.
status	String	No	Estado operativo actual de la mesa.	Valor por defecto: "Available". Valores posibles: "Available" / "Occupied".
seats	Number	Sí	Capacidad máxima de personas de la mesa.	Número entero positivo. Ej: 4, 6.
currentOrder	ObjectId	No	Referencia al pedido activo asignado a la mesa.	Clave foránea lógica → referencia al documento _id de la colección orders. Valor null cuando la mesa está disponible.

Tabla 24: Diccionario de datos: Colección Tables

Colección: payments Registra los comprobantes de las transacciones financieras procesadas, almacenando los identificadores de seguimiento del sistema de cobros y el estado de cada operación.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.
paymentId	String	No	Identificador de la transacción en el procesador.	ID retornado por el simulador o pasarela externa (ej. TXN_829312).
orderId	String	No	Identificador del pedido asociado al pago.	Relación lógica con la colección orders. Almacenado como String para compatibilidad.
amount	Number	No	Monto total de la transacción.	Valor numérico decimal. Representa el importe cobrado.
currency	String	No	Código de la moneda de la transacción.	Ej: "BOB", "USD".
status	String	No	Estado de la operación de pago.	Valores posibles: "Captured", "Pending", "Failed".
method	String	No	Canal o instrumento de pago utilizado.	Ej: "Cash", "Card", "QR", "Transfer".

Campo	Tipo	Requerido	Descripción	Notas Técnicas
email	String	No	Correo electrónico del pagador.	Utilizado para el envío del comprobante digital.
contact	String	No	Dato de contacto adicional.	Número de teléfono o contacto del cliente.
createdAt	Date	No	Fecha y hora de registro del pago.	Generado al momento de procesar el cobro exitoso.

Tabla 25: Diccionario de datos: Colección Payments

Colección: orders Constituye el eje central del sistema. Registra cada transacción de venta de forma integral, vinculando los datos del cliente, los productos consumidos, el resumen de facturación, la mesa asignada y el método de pago procesado.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.
customerDetails	Object	Sí	Datos del cliente para quien se abre el pedido.	Objeto anidado: name (String), phone (Number) y guests (Number).
orderStatus	String	Sí	Estado actual del pedido en el flujo de servicio.	Valores posibles: "Pending", "In Preparation", "Served", "Paid".

Campo	Tipo	Requerido	Descripción	Notas Técnicas
orderDate	Date	No	Fecha y hora de creación del pedido.	Valor por defecto: <code>Date.now()</code> .
bills	Object	Sí	Resumen de facturación calculado por el servidor.	Objeto anidado: <code>total</code> , <code>tax</code> y <code>totalWithTax</code> .
items	Array	No	Lista de productos incluidos en el pedido.	Arreglo de objetos con datos desnormalizados para garantizar la inmutabilidad histórica.
table	ObjectId	No	Mesa física asignada al pedido.	Referencia lógica → vinculada al <code>_id</code> de la colección <code>tables</code> .
paymentMethod	String	No	Método de pago utilizado para el cierre.	Ej: <code>"Cash"</code> , <code>"Digital Payment"</code> , <code>"QR"</code> .
paymentData	Object	No	Metadatos de confirmación de la transacción.	Almacena IDs de seguimiento generados por el simulador o pasarela externa (<code>transaction_id</code>).
createdAt	Date	Sí	Fecha y hora de registro del documento.	Generado automáticamente mediante timestamps de Mongoose.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
updatedAt	Date	Sí	Fecha y hora de la última modificación.	Actualizado automáticamente mediante timestamps de Mongoose.

Tabla 26: Diccionario de datos: Colección Orders

3.5.3 Colección: dishes

Almacena la información detallada de los platillos, productos o bebidas disponibles en el menú del restaurante.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del platillo.	Generado automáticamente por MongoDB.
name	String	Sí	Nombre comercial del plato o bebida.	Debe ser único para evitar duplicados en el menú.
price	Number	Sí	Precio de venta al público.	Se almacena como valor numérico (decimal/flotante).
category	ObjectId	Sí	Referencia a la categoría del plato.	Vinculado a la colección <i>categories</i> mediante <i>Population</i> .

Campo	Tipo	Requerido	Descripción	Notas Técnicas
type	String	No	Clasificación del tipo de producto.	Valor por defecto: "General". Permite agrupar por etiquetas personalizadas.

Tabla 27: Diccionario de datos: Colección Dishes

3.5.4 Colección: categories

Esta colección clasifica los productos (ej. Entradas, Platos Fuertes, Bebidas) para facilitar la navegación en el punto de venta y agrupar los platillos por familias.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único de la categoría.	Generado automáticamente por MongoDB.
name	String	Sí	Nombre de la categoría.	Indexado como unique: true para evitar duplicidad de nombres.
bgColor	String	No	Color de fondo para la interfaz visual.	Valor por defecto: "#b73e3e". Almacenado en formato Hexadecimal.
icon	String	No	Icono o emoji representativo.	Valor por defecto: "[icono]". Se utiliza para la identificación rápida en el frontend.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
-------	------	-----------	-------------	----------------

Tabla 28: Diccionario de datos: Colección Categories

3.5.5 Relaciones entre colecciones

El modelo de datos, aunque documental (NoSQL), establece relaciones lógicas explícitas entre las colecciones mediante referencias `ObjectId`, preservando la integridad referencial del sistema TPS:

Relación	Cardinalidad	Descripción
<code>users → orders</code>	1 : N	Un usuario (cajero) puede haber procesado múltiples órdenes a lo largo de su operación. Cada orden registra implícitamente al operador responsable de la sesión activa.
<code>tables → orders</code>	1 : N	Una mesa puede estar asociada a múltiples órdenes a lo largo del tiempo (una por cada servicio). En un momento dado, solo una orden puede estar activa por mesa (<code>currentOrder</code>).
<code>tables → orders (activa)</code>	1 : 1	En tiempo real, la relación entre una mesa y su pedido en curso es uno a uno: el campo <code>currentOrder</code> en <code>Table</code> apunta a exactamente un único documento activo en <code>orders</code> .

Relación	Cardinalidad	Descripción
orders → payments	1 : 1	Cada orden pagada genera exactamente un registro de pago en la colección payments. La relación se establece a través del campo orderId en Payment.

Tabla 29: Relaciones y Cardinalidad del Modelo de Datos

3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA

3.6.1 Módulo de Gestión de Procesos

Descripción y propósito El Módulo de Gestión de Procesos constituye el **cimiento operativo** del Sistema POS. Su responsabilidad es la administración de las **entidades maestras** del negocio: el catálogo de **Productos** (el menú de la cafetería) y el inventario de **Mesas** (la infraestructura física del establecimiento). Sin datos correctamente cargados en estas dos entidades, el módulo transaccional del POS no puede operar: no es posible construir una orden sin productos en el menú, ni asignarla a una mesa si estas no están registradas en el sistema.

El acceso a este módulo está **restringido exclusivamente al rol Administrador**, ya que la creación, modificación o eliminación de estos datos maestros afecta directamente la operativa de todos los cajeros del turno.

Submódulo: Gestión del Catálogo de Productos Este submódulo provee las interfaces y los *endpoints* necesarios para mantener actualizado el menú digital de la cafetería. Implementa las cuatro operaciones CRUD completas sobre la colección products de MongoDB:

- **Creación (Create):** El Administrador accede al formulario de alta de producto

en el panel de administración de React. Ingresas los atributos del ítem (nombre, descripción, precio, categoría —ej. "Cafés", "Infusiones", "Postres", "Snacks"— e imagen). El *frontend* envía una petición POST `/api/products` al backend, que valida los datos contra el esquema Mongoose del `ProductSchema` antes de insertar el nuevo documento en MongoDB.

- **Lectura (Read):** La interfaz POS del cajero consume el *endpoint* GET `/api/products` para cargar el catálogo de productos activos, organizados por categoría, que se muestran en la grilla de selección táctil. El Administrador puede consultar el listado completo, incluyendo productos inactivos, desde el panel de gestión.
- **Actualización (Update):** El Administrador puede modificar cualquier atributo de un producto existente (incluyendo su precio) mediante una petición PUT `/api/products/:id`. **Restricción clave de integridad histórica:** la actualización de precios solo afecta a futuras órdenes. Los documentos de órdenes ya cerradas preservan el precio exacto del momento de la venta, gracias a la desnormalización controlada del campo `items` en la colección `orders`.
- **Eliminación lógica (Delete):** En lugar de borrar físicamente el registro, el sistema implementa una **eliminación lógica** mediante un cambio de estado (`active: false`). Esto garantiza que los productos que ya forman parte del historial de ventas sigan siendo referenciables en los reportes, sin aparecer en el menú activo del POS.

Submódulo: Gestión de Mesas Este submódulo administra el registro de las mesas físicas del establecimiento, que son los nodos de anclaje del flujo transaccional del POS. Opera sobre la colección `tables` de MongoDB:

- **Alta de mesa (Create):** El Administrador registra una nueva mesa especificando su número (`tableNo`, único en el sistema) y su capacidad en asientos (`seats`). La mesa se crea con estado inicial "Available" y sin orden activa (`currentOrder: null`). Endpoint: POST `/api/tables`.

- **Consulta del panel de mesas (Read):** Los cajeros y el Administrador acceden al *endpoint* GET `/api/tables` para renderizar el panel visual de estados, que muestra en tiempo real qué mesas están disponibles ("Available") y cuáles están ocupadas con un pedido en curso ("Occupied").
- **Actualización de estado (Update):** El estado de una mesa es actualizado automáticamente por el backend durante el flujo transaccional: se marca como "Occupied" al abrir una orden, y vuelve a "Available" al confirmar el pago y cerrar la transacción. El Administrador también puede actualizar manualmente el estado o los datos de una mesa a través de PUT `/api/tables/:id`.
- **Baja de mesa (Delete):** La eliminación de una mesa del registro solo está permitida si no tiene una orden activa vinculada (`currentOrder: null`), preservando la integridad referencial del historial.

3.6.2 Módulo de Usuarios y Roles

Descripción y propósito El Módulo de Usuarios y Roles constituye la **barrera de seguridad** del Sistema TPS. Su función es garantizar que cada actor que interactúe con el sistema sea quien dice ser (*autenticación*) y que solo pueda ejecutar las acciones para las que tiene autorización según su rol (*autorización*). Este módulo implementa un modelo de **Control de Acceso Basado en Roles** (RBAC — *Role-Based Access Control*), diferenciando dos perfiles funcionales: "Admin" y "Cashier".

Mecanismo de autenticación: JSON Web Tokens (JWT) El sistema descarta el uso de sesiones basadas en servidor (*server-side sessions*) —que requieren almacenamiento de estado en el backend y presentan problemas de escalabilidad horizontal— en favor de **autenticación sin estado (*stateless*) mediante JWT** (Jones et al., 2015).

El flujo de autenticación opera de la siguiente manera:

1. **Solicitud de acceso:** El operador ingresa su correo y contraseña en la pantalla de *login* de React. El *frontend* envía una petición POST `/api/auth/login` al

backend con las credenciales en el cuerpo del *request*.

2. **Verificación de identidad:** El backend localiza el documento del usuario en la colección `users` a partir del correo electrónico. Utilizando la función `bcrypt.compare()`, compara la contraseña recibida (texto plano) con el *hash* almacenado en la base de datos. Gracias al diseño de Bcrypt, esta comparación es segura incluso ante ataques de fuerza bruta, ya que el proceso de *hashing* con un factor de coste de 10 hace computacionalmente costosa la verificación masiva de contraseñas.
3. **Emisión del token:** Si las credenciales son válidas, el backend genera un **JSON Web Token** firmado con una clave secreta (`JWT_SECRET`) almacenada como variable de entorno. El *payload* del token contiene el `_id` del usuario y su role ("Admin" o "Cashier"), con un tiempo de expiración configurado (ej. "8h", equivalente a un turno de trabajo).
4. **Uso del token:** El *frontend* almacena el JWT en el estado global de Redux (y opcionalmente en `localStorage`). A partir de ese momento, **cada petición HTTP al backend incluye el token en la cabecera** `Authorization: Bearer <token>`.
5. **Validación en cada petición:** Un *middleware* de Express (`verifyToken`) intercepta todas las rutas protegidas antes de ejecutar el controlador correspondiente. Deserializa el token usando `jwt.verify()`, extrae el *payload* y lo adjunta al objeto `req.user`, haciendo disponibles la identidad y el rol del solicitante para los *middlewares* de autorización subsiguientes.

Mecanismo de protección de contraseñas: Bcrypt El almacenamiento de contraseñas en texto plano es una vulnerabilidad crítica inaceptable en cualquier sistema de producción. El sistema implementa **Bcrypt** como función de *hashing* adaptativa de contraseñas (Provos & Mazières, 1999):

- **Proceso de registro:** Al crear un nuevo usuario, el backend ejecuta

`bcrypt.hash(password, 10)` antes de persistir el documento en MongoDB. Este proceso: (a) genera un *salt* criptográficamente aleatorio, (b) concatena el *salt* con la contraseña, y (c) aplica el algoritmo Blowfish iterativamente $2^{10} = 1.024$ veces, produciendo un *hash* de 60 caracteres que incluye el *salt* incrustado. La contraseña original nunca se almacena ni se puede recuperar.

- **Factor de coste adaptativo:** El valor 10 representa el factor de coste (número de rondas de *hashing*). Este parámetro puede incrementarse en futuras versiones del sistema para compensar el aumento de la potencia computacional, manteniendo el algoritmo resistente a ataques por diccionario y fuerza bruta sin modificar el código.

Diferenciación de permisos por rol Los dos roles del sistema tienen accesos claramente delimitados, implementados mediante *middlewares* de autorización en el backend y renderizado condicional en el frontend:

Acción / Recurso	Rol Admin	Rol Cashier
Iniciar sesión	SI	SI
Operar el módulo POS (crear y cerrar órdenes)	SI	SI
Consultar estado de mesas	SI	SI
Gestionar catálogo de productos (CRUD)	SI	NO
Gestionar mesas (CRUD)	SI	NO
Crear, editar o eliminar usuarios	SI	NO
Acceder al módulo de reportes financieros	SI	NO

Acción / Recurso	Rol Admin	Rol Cashier
Consultar historial completo de ventas	SI	NO
Ver únicamente las ventas de su turno	SI	SI

Tabla 30: Matriz de Control de Acceso por Roles (RBAC)

Implementación en el backend: Un segundo *middleware* de Express (`verifyRole('Admin')`) se encadena después de `verifyToken` en las rutas sensibles. Si el `req.user.role` no coincide con el rol requerido, el *middleware* interrumpe la cadena y retorna una respuesta 403 Forbidden con un mensaje de error descriptivo, sin ejecutar el controlador.

Implementación en el frontend: React renderiza condicionalmente los componentes de navegación y las opciones del menú lateral basándose en el rol almacenado en el estado de Redux. Un cajero nunca verá los botones de administración de productos ni el acceso al panel de reportes gerenciales, reduciendo la superficie de error operativo y mejorando la experiencia de usuario.

3.6.3 Módulo de Transacciones

Núcleo central del sistema TPS para la cafetería, diseñado en React.js para ofrecer una interfaz táctil de alta velocidad y baja fricción en la toma de pedidos (Punto de Venta).

- **Toma de Pedidos (POS):** Interfaz interactiva para seleccionar categorías (Cafés, Infusiones, Postres, Snacks) y agregar productos al carrito de compras con sus respectivas cantidades.
- **Gestión de Mesas y Estados:** Vinculación obligatoria de cada orden a una mesa específica de la cafetería. Control del flujo del pedido cambiando su estado: “En preparación” (barra/cocina), “Servido” y “Pagado”.

- **Procesamiento de Pago y Cierre:** Cálculo automático en tiempo real de subtotales, impuestos y total a cobrar. Registro del método de pago (efectivo, tarjeta) e impresión del comprobante o ticket de venta.
- **Historial de transacciones:** Bitácora inmutable en MongoDB de todas las ventas realizadas, asociadas al cajero en turno, con protección contra alteraciones concurrentes.

3.6.4 Módulo de Reportes

Módulo analítico estadístico que destila la información transaccional operativa de la cafetería para facilitar la toma de decisiones de la gerencia.

- **Dashboard Estadístico:** Panel visual en el *frontend* que emplea librerías de gráficos (ej. Chart.js o Recharts) para mostrar las métricas clave en tiempo real.
- **Reportes de Ventas:** Consultas agregadas a MongoDB para extraer ingresos diarios, semanales o mensuales.
- **Rendimiento de Productos:** Identificación automática de los productos más vendidos (ej. Capuchino, Croissants) y los de menor rotación en el menú.
- **Exportación de Datos:** Capacidad para generar y descargar los reportes consolidados por cajero o por turnos en formatos limpios como PDF o Excel, facilitando el arqueo de caja y la contabilidad externa.

3.7 Capa Backend Funcional

El *backend* de la cafetería está desarrollado en **Node.js** con el *framework* **Express.js**, actuando como una API RESTful robusta, aislada de la interfaz gráfica y conectada a **MongoDB**. Su arquitectura sigue el patrón MVC (Modelo-Vista-Controlador) adaptado a servicios:

- **Conexión BD:** Implementación de la cadena de conexión segura hacia MongoDB utilizando la librería mongoose para el modelado de datos mediante esquemas estructurados.

- **Modelos (Models):** Representación orientada a documentos de las entidades principales de la cafetería: `UserSchema` (personal operativo y administrativo), `ProductSchema` (menú), `TableSchema` (mesas) y `OrderSchema` (transacciones/ventas).
- **Controladores (Controllers):** Funciones que capturan las peticiones HTTP (GET, POST, PUT, DELETE), procesan la lógica central de ventas y devuelven respuestas estandarizadas en formato JSON.
- **Rutas y Servicios (Routes/Services):** Definición ordenada de los *endpoints* de la API (`/api/orders`, `/api/products`, etc.) extrayendo la lógica de negocio a un nivel de servicio para mantener controladores limpios.
- **Capa de Seguridad (Middlewares):** Bloques intermedios que protegen rigurosamente las rutas. Incluyen la verificación de autenticidad mediante la validación y descryptación de JSON Web Tokens (JWT) y la autorización por niveles (Ej. bloqueando a un Cajero de la ruta de borrado de productos).
- **Validaciones:** Uso de librerías en el *backend* para verificar la integridad de los *payloads* antes de interactuar con la base de datos (ej. asegurar que una orden recibida contenga obligatoriamente el ID de una mesa válida y al menos un producto).

3.8 Validación y pruebas del sistema

El sistema asegura la calidad del producto final a través de distintas evaluaciones de estrés y rendimiento:

- **Pruebas unitarias:** Validando que las funciones matemáticas y servicios individuales del *backend* operen según la lógica.
- **Pruebas funcionales:** Ejecución de casos de uso (Ej: Qué sucede si el usuario ingresa un formato de fecha erróneo).
- **Pruebas de integración:** Ensayos del flujo Cliente hacia la API y hacia la base de datos extremo a extremo.
- **Pruebas de aceptación:** Pruebas finales realizadas con un entorno cercano a la

organización para validación definitiva del *Product Owner*.

3.9 Desarrollo del prototipo funcional

A lo largo de los *sprints* se generan prototipos incrementales. En esta etapa el proyecto expone sus interfaces plenamente interactivas reflejando casos de éxito desde el inicio de sesión (*login*), hasta el registro exitoso de la operación. (*Incluir evidencias y capturas de pantalla reales aquí*).

3.10 Documentación de Ingeniería Completa

Se acompañan como anexos técnicos o repositorios vinculados:

- **Documentación funcional**

La documentación funcional consolida todos los artefactos producidos durante la fase de análisis y modelado del sistema. Está orientada a servir como referencia oficial para el equipo de desarrollo, el *Product Owner* y las pruebas de aceptación.

- **Especificación de Requerimientos de Software (SRS)**

La Especificación de Requerimientos de Software documenta de forma estructurada y completa todos los requerimientos identificados para el Sistema POS de Cafetería. Se organiza en las siguientes secciones:

1. **Propósito del sistema:**

Desarrollar un Sistema de Punto de Venta (POS) web basado en la arquitectura MERN para digitalizar y centralizar las operaciones de una cafetería en la ciudad de La Paz, eliminando los procesos manuales y garantizando la trazabilidad de cada transacción.

2. **Alcance funcional:**

El sistema cubre seis módulos funcionales:

- Gestión de Ventas y Pedidos (RF-01, RF-08, RF-09, RF-11, RF-12, RF-13, RF-14)

- Control de Inventario (RF-03, RF-15, RF-16, RF-17, RF-18, RF-19)
- Gestión de Menú y Productos (RF-05, RF-06, RF-07, RF-10, RF-20, RF-21)
- Reportes y Toma de Decisiones (RF-04, RF-22, RF-23, RF-24, RF-25)
- Gestión de Proveedores y Compras (RF-02, RF-26, RF-27, RF-28)
- Gestión de Personal y Turnos (RF-29, RF-30, RF-31, RF-32)

3. Restricciones del sistema:

- Plataforma exclusivamente web
- Autenticación interna sin proveedores OAuth
- Pagos simulados sin integración bancaria real
- Sistema monosucursal en su primera versión
- Sin descuento automático de insumos compuestos

(Referencia completa en la sección Límites y Alcances del Marco Referencial).

4. Requerimientos funcionales:

32 requerimientos funcionales organizados en 6 módulos, con identificador, descripción, actor responsable y prioridad.

(Referencia: Tablas RF-01 a RF-32 en la sección 3.2).

5. Requerimientos no funcionales:

13 requerimientos no funcionales en las categorías de Rendimiento, Confiabilidad, Seguridad, Disponibilidad, Escalabilidad, Usabilidad y Mantenibilidad.

(Referencia: Tabla RNF-01 a RNF-13 en la sección 3.2).

• Relevamiento documentado

El relevamiento de información se realizó mediante las siguientes técnicas aplicadas al personal de la cafetería:

1. Entrevista estructurada al administrador:

Se identificaron los procesos críticos de cobro, control de inventario y generación de reportes. El administrador manifestó la necesidad urgente de eliminar los descuadres de caja diarios y contar con información de ventas en tiempo real.

2. Observación directa del flujo de servicio:

Se documentó el ciclo completo de atención al cliente:

toma de pedido verbal → comanda en papel → preparación → cobro manual → anotación en cuaderno

Se identificaron los puntos de falla más frecuentes: comandas ilegibles, errores de suma y ausencia de trazabilidad.

3. Revisión del repositorio de referencia:

Se analizó la estructura del repositorio *Restaurant_POS_System* (amritmaurya1504, GitHub), identificando los módulos implementados: gestión de órdenes en tiempo real, reservas de mesa, autenticación con control de roles, integración de pagos y facturación automática. Estos módulos sirvieron como base para la definición del alcance funcional del presente sistema.

- **Documentación técnica:** El diagrama de la arquitectura desplegada, diccionarios de datos, modelo E/R completo y especificación paramétrica de API.
- **Documentación del sistema:** Manual de usuario para operadores, el manual técnico, directrices de instalación en entorno de servidor y parametrización de variables de entorno.
- **Documentación del código:** Detalla la estructura de directorios del proyecto MERN y las dependencias utilizadas en el sistema de la cafetería.
 - **Estructura del Proyecto:** Separación física y lógica entre el cliente (aplicación React en el directorio /frontend) y el servidor (API Node.js en el directorio /backend).
 - **Librerías y Dependencias Backend:** Uso de express para el enrutamiento HTTP, mongoose como ODM para modelar los datos de MongoDB, jsonwebtoken para la generación y firma de tokens de sesión, bcryptjs para el hash de contraseñas, y cors para habilitar peticiones seguras desde el frontend.
 - **Librerías y Dependencias Frontend:** Uso de react para la construcción de interfaces de usuario interactivas del POS, react-router-dom para la

navegación entre el terminal de cobro y el panel de administración, y `axios` para consumir las rutas RESTful del backend asíncronamente.

Referencias Bibliográficas

- Brown, S. (2018). *Software architecture for developers: Visualising, documenting and exploring your software architecture*. Leanpub.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentos de sistemas de bases de datos* (7th ed.). Pearson Educación.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON web token (JWT)* (RFC No. 7519). Internet Requests for Comments; RFC Editor. <https://doi.org/10.17487/RFC7519>
- Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de información gerencial: Administración de la empresa digital* (16th ed.). Pearson Educación.
- O'Brien, J. A., & Marakas, G. M. (2011). *Sistemas de información gerencial* (10th ed.). McGraw Hill.
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. *Proceedings of the 1999 USENIX Annual Technical Conference*, 81–92.
- Schwaber, K., & Sutherland, J. (2020). *La guía definitiva de scrum: Las reglas del juego*. Scrum.org.
- Sommerville, I. (2015). *Ingeniería de software* (10th ed.). Pearson Educación.
- Stallings, W. (2017). *Fundamentos de seguridad en redes: Aplicaciones y estándares* (6th ed.). Pearson Educación.